

CareFlow Health Management System
CSI-4999 Senior Capstone - Jawad
March 9th, 2026

Group #2:
Tessa Esman
Mara Evans
Manuel Gardea
Zakaria Hoque
Logan Templin

Video link https://youtu.be/LS3n6_oGxro

Table of Contents

Project Description: 3
Target Audience: 3
Similar Existing System: 3
Revenue Streams: 3
User Types: 3
Software Architecture 4
Subsystems and Features (12): 4
Technology Used 5
Use Case Diagrams: 5
Use Case Vs SQL Query (Table 1) 10
(Figure 6) 13
System Sequence Diagrams (SSD): 13
Activity Diagrams: 15
Zakaria Hoque (Upload File) 15
System Sequence Diagrams (SSD): 16
Manuel Gardea (Delete File) 16
Activity Diagrams: 17
Activity Diagram (Add new Patient): 19
User Interface Design 24
Old Wireframe: 24
Old/Alternative UI Mockups with Data: 29
HCI Rules Discussion: 30
Final Screenshots: 32
Database Design: 55
ERD: 55
Database Tables: 56
ClinicLocation: 56
Bill: 56
Sample Code/Explanations: 58
Appendix A: System Vision Document 70
Appendix B: Work Breakdown Structure and Gantt Chart: 71

Project Description:
This project aims to create a healthcare management system intended to simplify operations across multiple medical locations. The system centralizes the management of clinics, staff, patients, billing, and medical records into a single secure platform. By replacing fragmented and paper-based workflows, the application aims to improve organizational efficiency, data accuracy, and accessibility for healthcare providers. The system is designed to support role-based access, scalable usage across multiple locations, and secure handling of sensitive patient information, making it suitable for small clinics, private practices, and healthcare organizations.

Target Audience:
The primary target audience for this healthcare management system includes small medical clinics, private medical practices, and larger healthcare organizations. These users would benefit from a centralized platform to manage multiple clinic locations, personnel, patient records, billing, and documentation. The system is especially suited for organizations seeking to replace manual hard copy-based paperwork with a digital system.

Similar Existing System:
CPR is a platform designed to replace paper charts by capturing, storing, and retrieving patient health information across care networks. Our system will allow users to manage multiple locations as well as information.

In contrast, our healthcare management system is designed as a broader operational platform. In addition to managing patient records, it supports multi-location clinic management, staff assignment, billing, payments, and document handling within a single integrated system. This approach allows healthcare organizations to manage both administrative and clinical workflows together

Revenue Streams:
The healthcare management system will generate revenue through a subscription-based model offered on a monthly or yearly basis. Pricing will be tiered based on the number of staff members and patient users supported by the system, allowing organizations of different sizes to select a plan that fits their needs. Additional revenue may be generated through optional add-on features, such as expanded storage, advanced reporting, or enhanced billing tools, providing flexibility and scalability for growing healthcare organizations.

User Types:
There are two basic users, staff and admin. The staff user will function as a standard healthcare worker, such as a nurse or billing. These users have access to the patient management, billing and records subsystems. The admin users on the other hand, are meant to function as higher

personnel, such as account managers. As such, the admins have access to the previous systems, as well as clinic management and staff management.

Software Architecture

Subsystems and Features (12):

The healthcare management system is divided into five core subsystems. Each subsystem focuses on a specific operational area and includes multiple features that interact directly with a database through distinct use cases. These subsystems include *Location Management, Patient Management, Staff Management, Billing Information*, and *Records Management*. Together, they provide support for managing clinic locations, personnel, patient data, billing processes, and medical documentation, resulting in a system with over twenty distinct use cases.

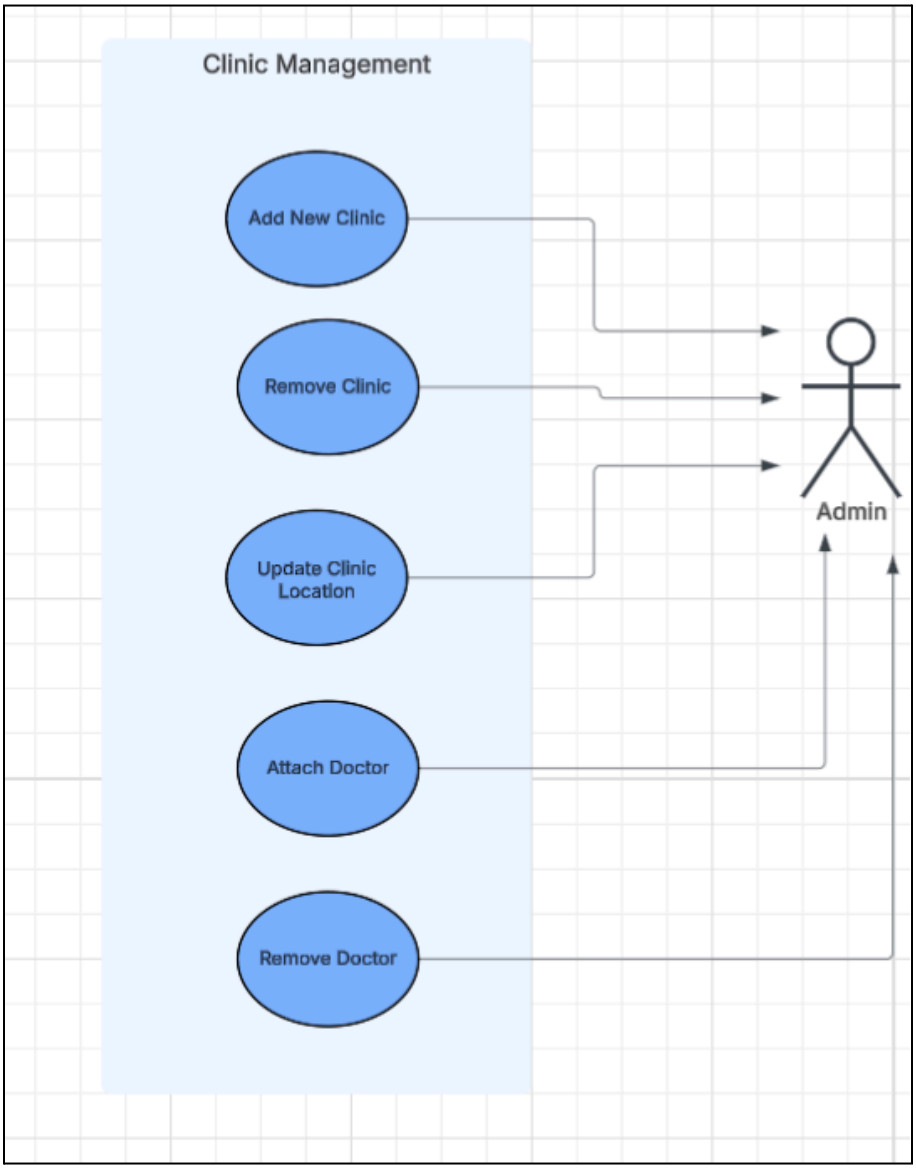
(The highlighted use cases are completed)

- **Location Management**
  - Add new Clinic Locations
  - Remove (old) Clinic Locations
  - Update Clinic Locations
- **Patient Management**
  - Add patient
  - Remove patient
  - Add patient information
  - Update patient information
  - Delete patient information
- **Staff Management**
  - Add staff
  - Remove staff
  - Update staff information
  - Delete staff information
- **Billing Information**
  - Add bills
  - Pay bill
  - Add payment method
  - Remove payment method
- **Records**
  - Upload document
  - Download document
  - Delete document
  - Search for Document

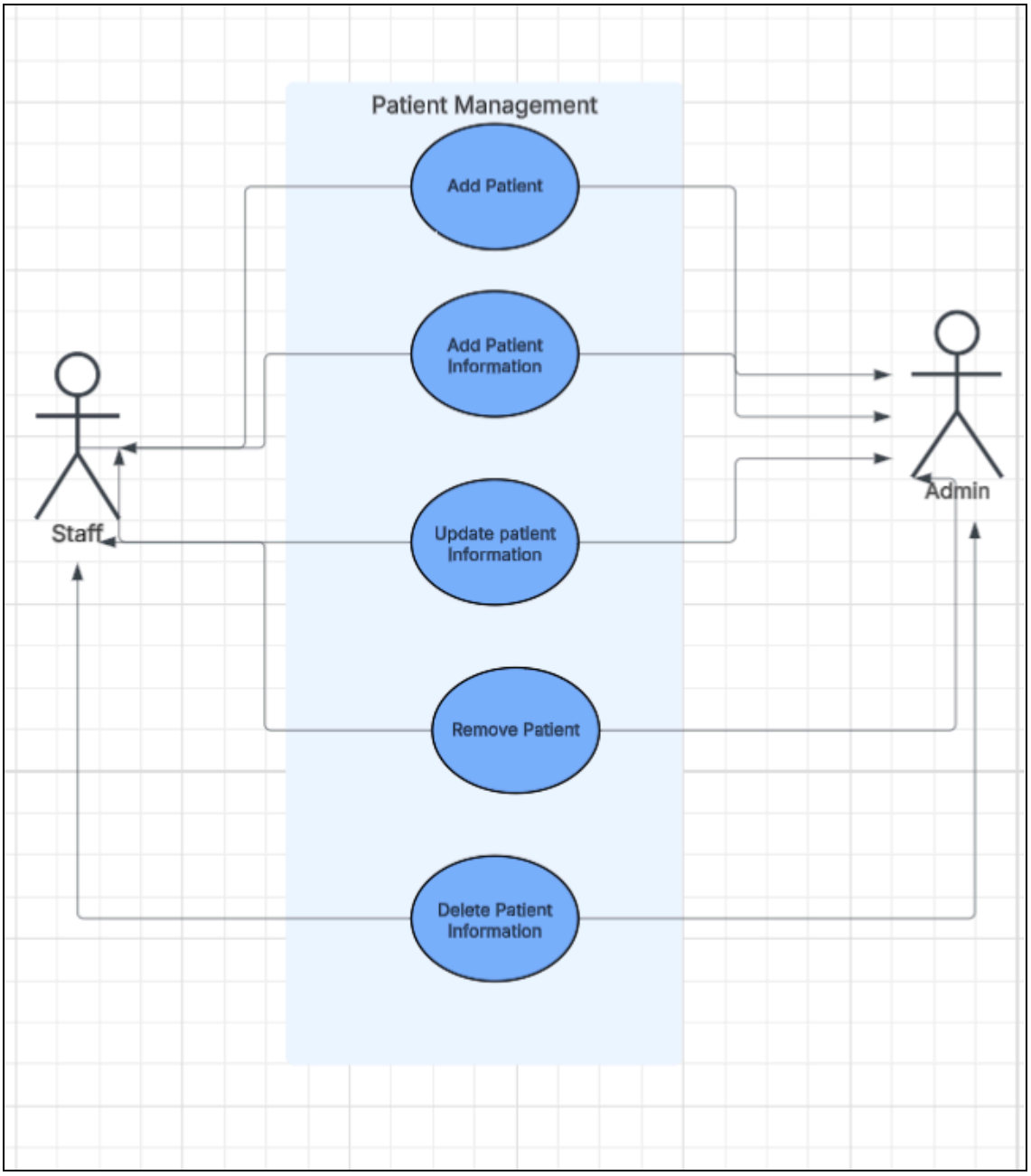
Technology Used

The program uses the python coding language and a SQLite database, as python has a built-in library for this backend language. The GUI for the frontend utilizes the tkinter visual library, the currently most popular GUI development library for python. Other libraries used include the os library and filedialog library. The software is being developed using the VScode IDE. To create a codebase for us to share and merge our work, we are using GitHub.

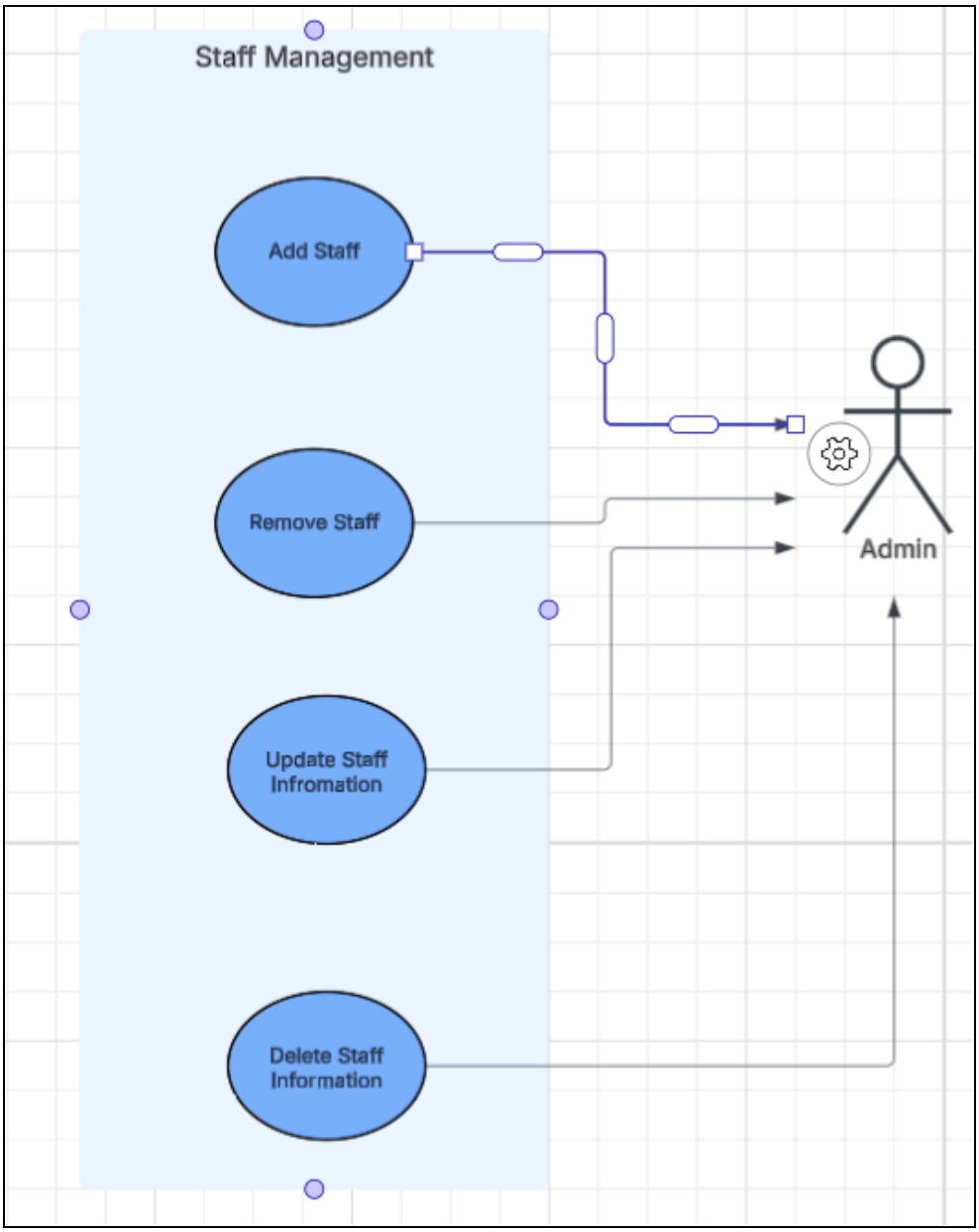
Use Case Diagrams:



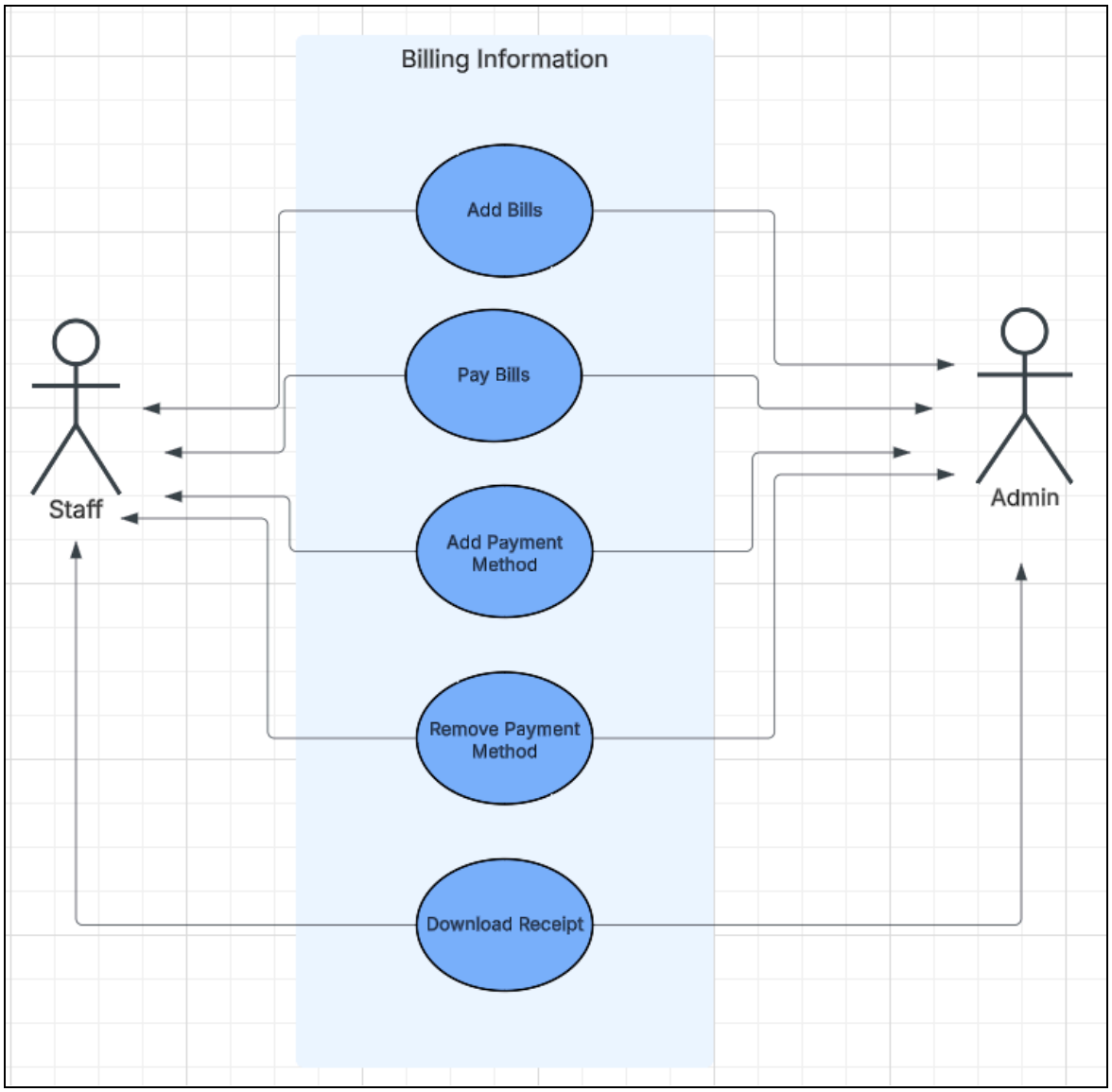
(Figure 1) Clinic Management Use Case Diagram



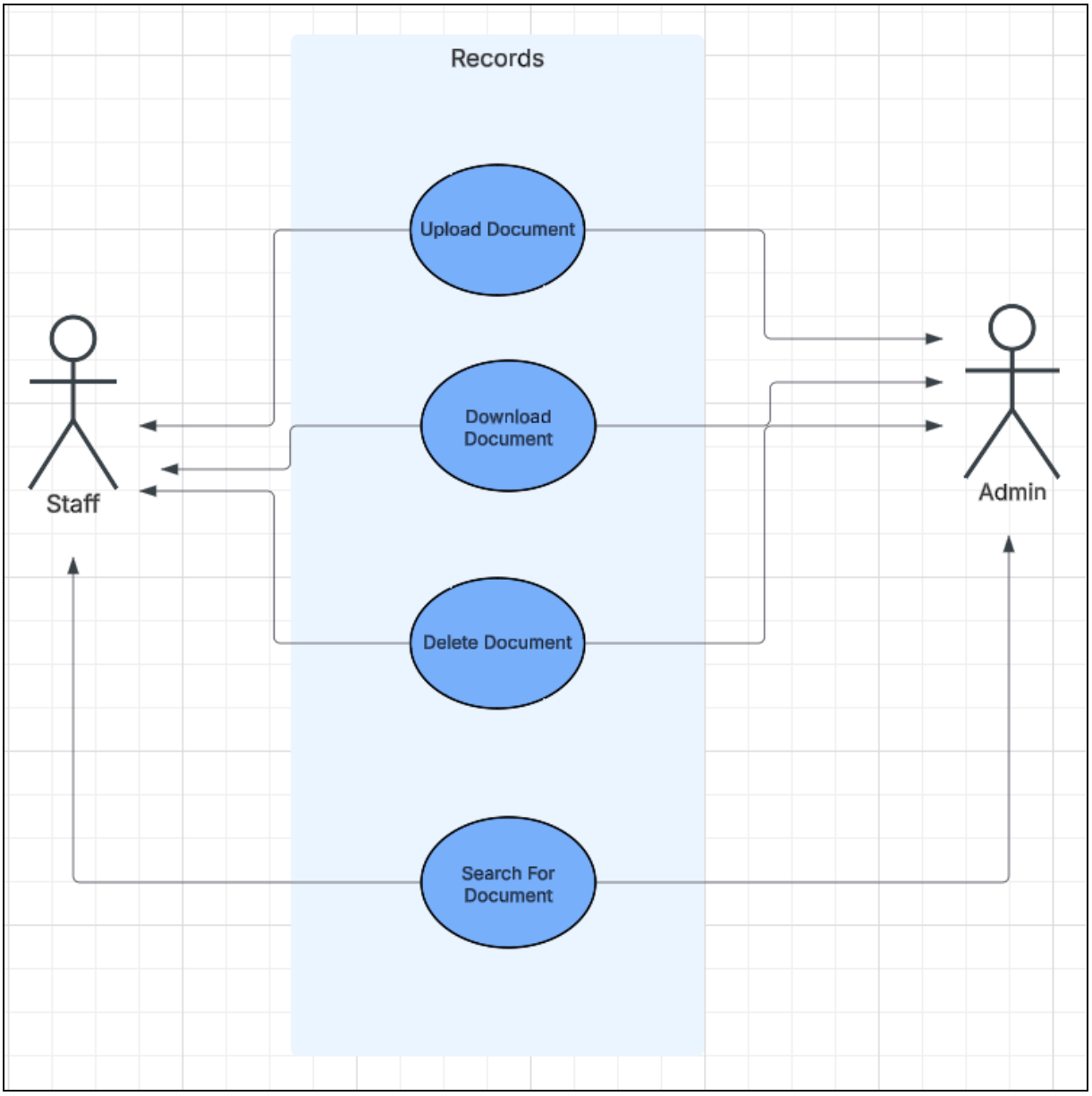
(Figure 2): Patient Management Use Case Diagram



(Figure 3) Staff Management Use Case Diagram



(figure 4): Billing Information Use Case Diagram



(Figure 5) Record System Use Case Diagram

Use Case Vs SQL Query (Table 1)

#	Use Case	SQL Query	Table(s)	File	Description
1	Upload Document	"INSERT INTO records (patient_id, staff_id, filename, filepath, upload_date) VALUES (?, ?, ?, ?, ?)", (patient_id, staff_id, final_filename, dest_path, upload_date)	records	records.py	This allows users to upload a document file to the records directory. The button will pull up the file system, allowing the user to select the file to upload. The function will then take the file metadata and inserts it into the records table, tracking the filename, filepath and upload date, as well as tracking the patient id and staff id associated with the file.
2	Download Document	"SELECT filepath FROM records WHERE record_id = ?", (record_id,)	records	records.py	This allows users to download the file they have selected from the listbox. It finds the filepath using the table entry “filepath”. The filesystem will open and allow them to determine where they want to save the file to.
3	Delete Document	"SELECT filepath FROM records WHERE record_id = ?", (record_id,) "DELETE FROM records WHERE record_id = ?", (record_id,)	records	records.py	This allows users to select a file from the listbox and delete it from the record directory and the file’s entry and metadata from the records table.
4	Search for Document	SELECT record_id, filename, upload_date FROM records WHERE patient_id = ? ORDER BY	records	record.py	This allows users to type a filename in the searchbox at the top of the page. As the user types in the name, the listbox will refresh and

		upload_date DESC """", (patient_id,)) rows = cur.fetchall()			pull relevant file names from the records table.
5	Add New Clinic Location	INSERT INTO ClinicLocation (name, address, city, state, zip, phone, status) VALUES (?, ?, ?, ?, ?, ?, 'active') """", (name, address, city, state, zip_code, phone)	ClinicLocation	Clinic_Location.py	This allows users to add a new clinic location and relevant data (name, address, city, zip_code, phone) to the ClinicLocation table. A pop up form will appear with relevant fields to input data.
6	Update Clinic Location	"UPDATE ClinicLocation SET {', '.join(fields)} WHERE location_id = ?"	ClinicLocation	Clinic_Location.py	This allows users to change any clinic location information they need to update. A pop form appears with the current data and the user can append these and save it to the ClinicLocation table.
7	Delete/Remove Clinic Location	UPDATE ClinicLocation SET status = 'inactive' WHERE location_id = ? """", (clinic_id,))	ClinicLocation	Clinic_Location.py	This allows users to delete the clinic entry from the database. This will reflect in the GUI when the listbox refreshes.
8	Add Bills	""" INSERT INTO Bill (patient_id, location_id, amount, due_date, status) VALUES (?, ?, ?, ?, 'unpaid') """", (patient_id, location_id, amount, due)	Bill	Billing_Staff_app.py	This allows the user to create a new bill for a specific patient at a specific clinic. This also allows the user to set the amount the patient owes, as well as the due date.
9	Pay Bill	"UPDATE Bill SET status = 'paid' WHERE	Bill	Billing_Patient_app.py	This allows the user to pay a bill they have

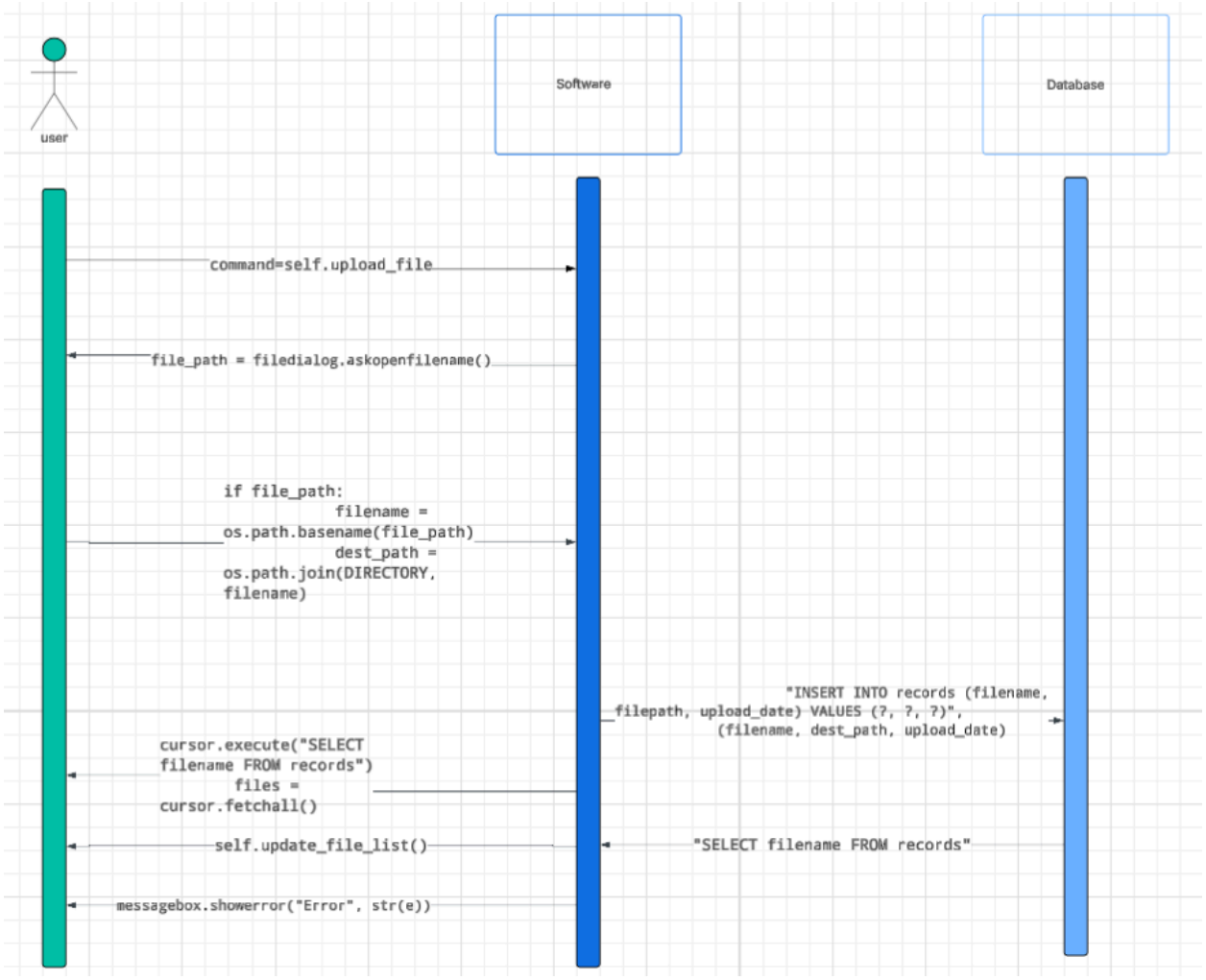
		bill_id = ?", (bill_id,)			assigned to them through the patient ID. The user must first fill out a payment method to pay the bill.
10	Add Patient	(""" INSERT INTO Patient (first_name, last_name, dob, sex, phone, email, address, location_id, allergies, conditions, medications, notes, emergency_contact, password_hash) VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?) """, (data["first_name"], data["last_name"], data["dob"], data["sex"], data["phone"], data["email"], data["address"], location_id, data["allergies"], data["conditions"], data["medications"], data.get("notes", ""),	Patient	Patient.py	The user can add a patient to the patient table. This will hold relevant patient info and history, such as name, date of birth, conditions, etc.

		data["emergency_contact"], pw_hash)			
11	Add Staff	""" INSERT INTO Staff (first_name, last_name, email, phone, role, active_flag, password_hash) VALUES (?, ?, ?, ?, ?, 1, ?) """, (data["first_name"], data["last_name"], data["email"], data["phone"], role, pw_hash)	Staff	Staff.py	The user can add a new staff member to a clinic. It asks for first name, last name, email, phone number, role (doctor, nurse,etc) and password. This will save them to the staff table. This requires a special staff code to be added.

(Figure 6)

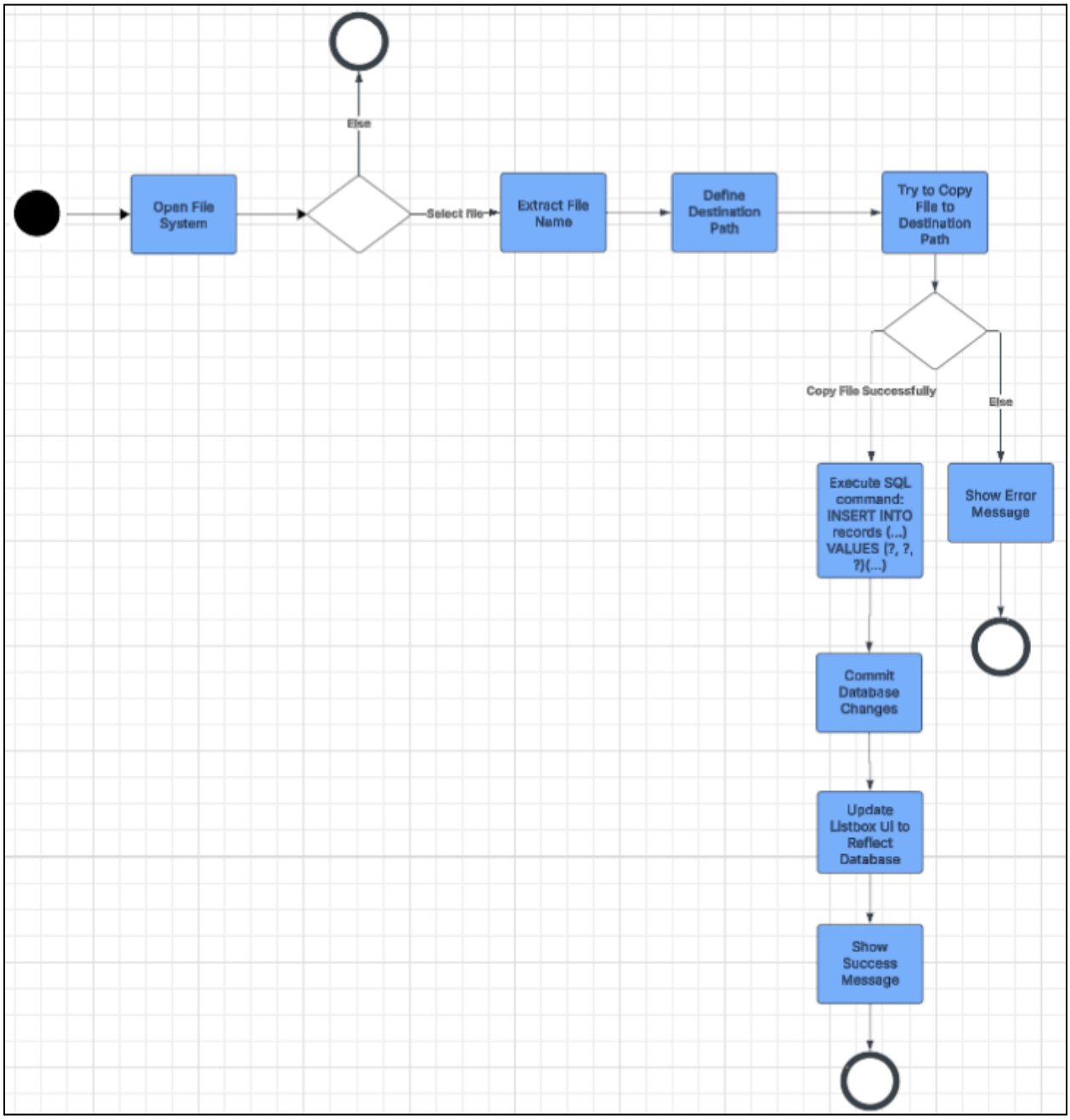
System Sequence Diagrams (SSD):

Zakaria Hoque (Upload File)



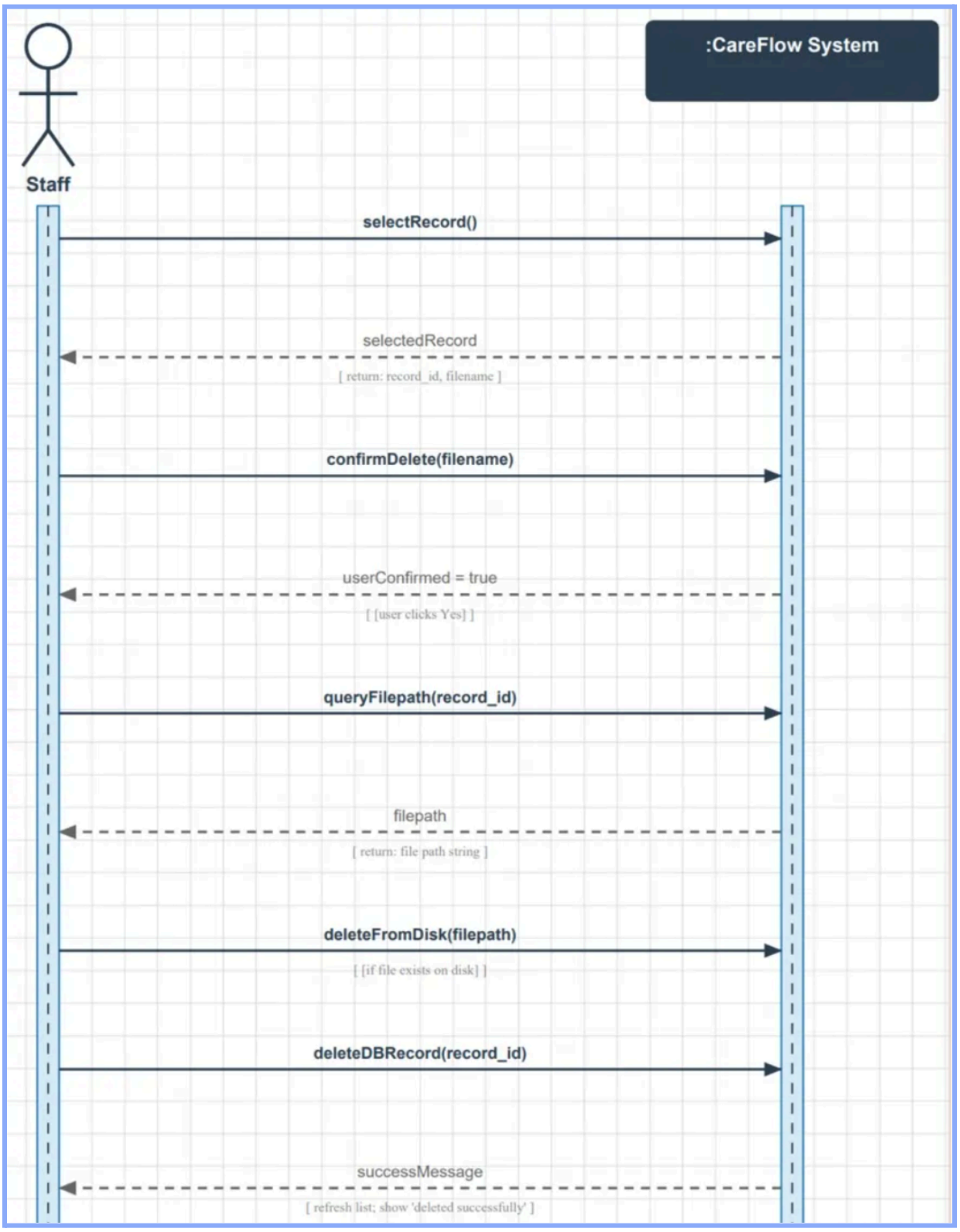
(Figure 7) Upload FIle SSD

Activity Diagrams:
Zakaria Hoque (Upload File)



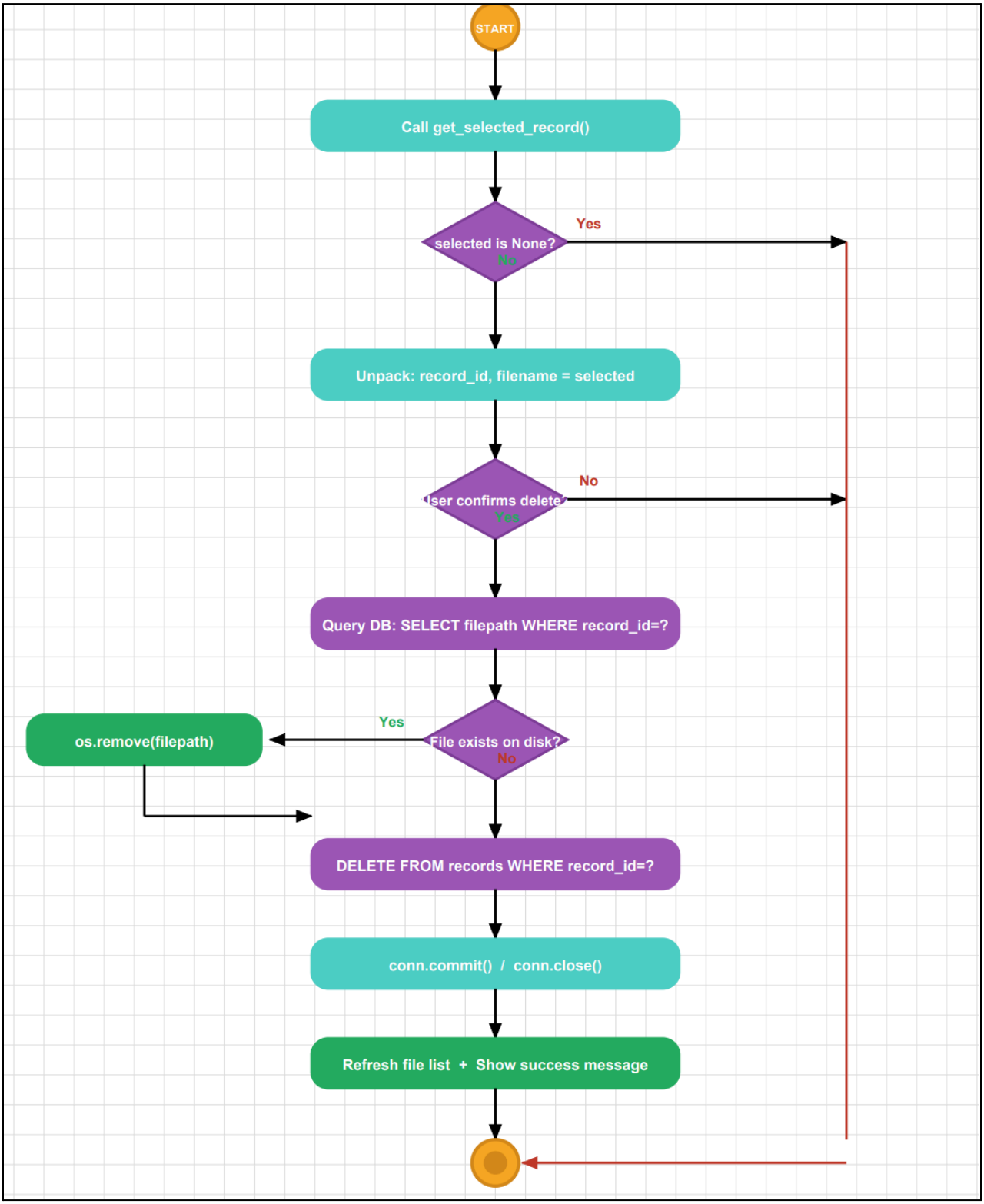
(Figure 8) Upload FIle Activity Diagram

System Sequence Diagrams (SSD):
Manuel Gardea (Delete File)



(Figure 9) Delete File SSD Diagram

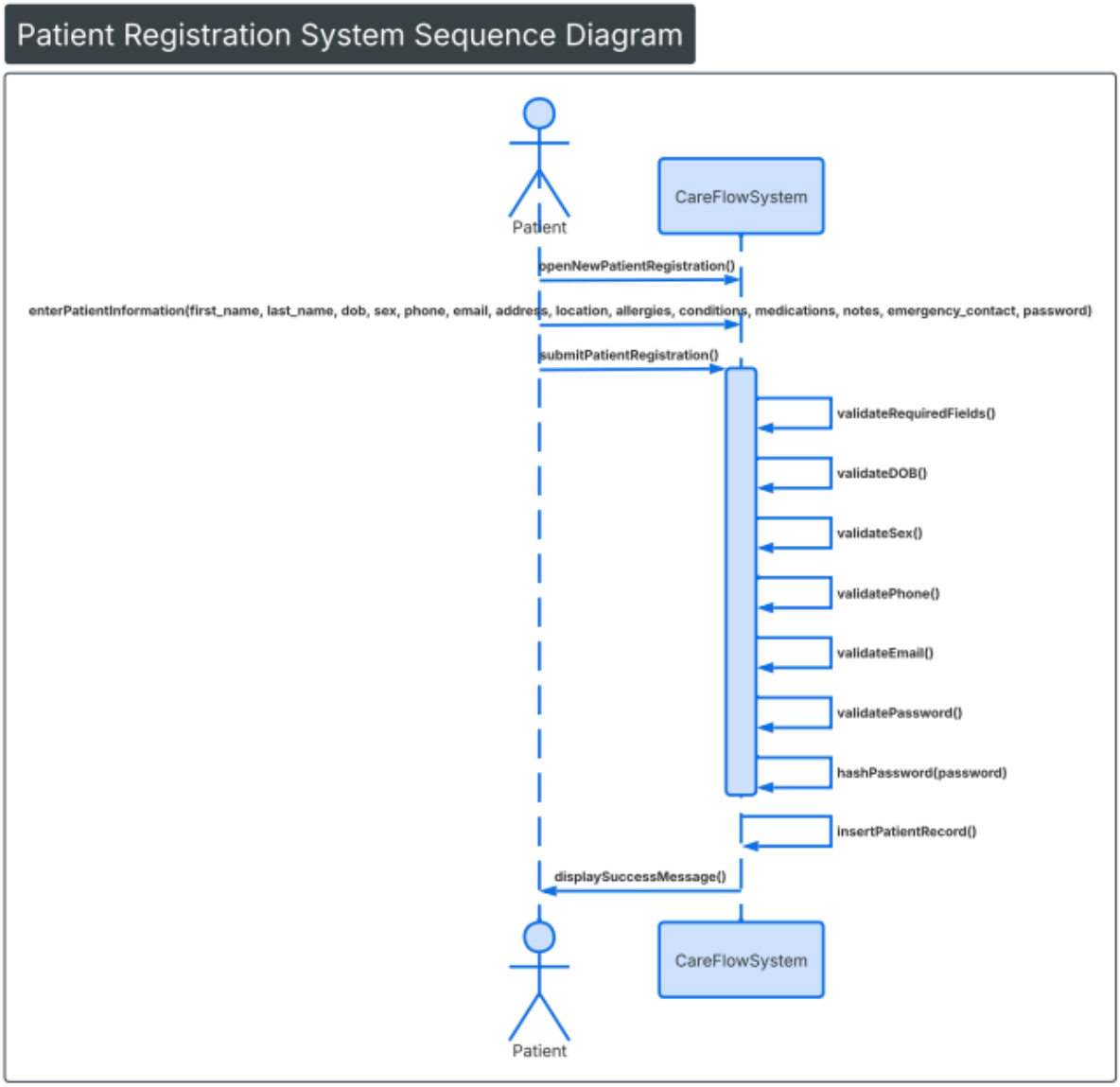
Activity Diagrams:
Manuel Gardea



(Figure 10) Delete File Activity Diagram

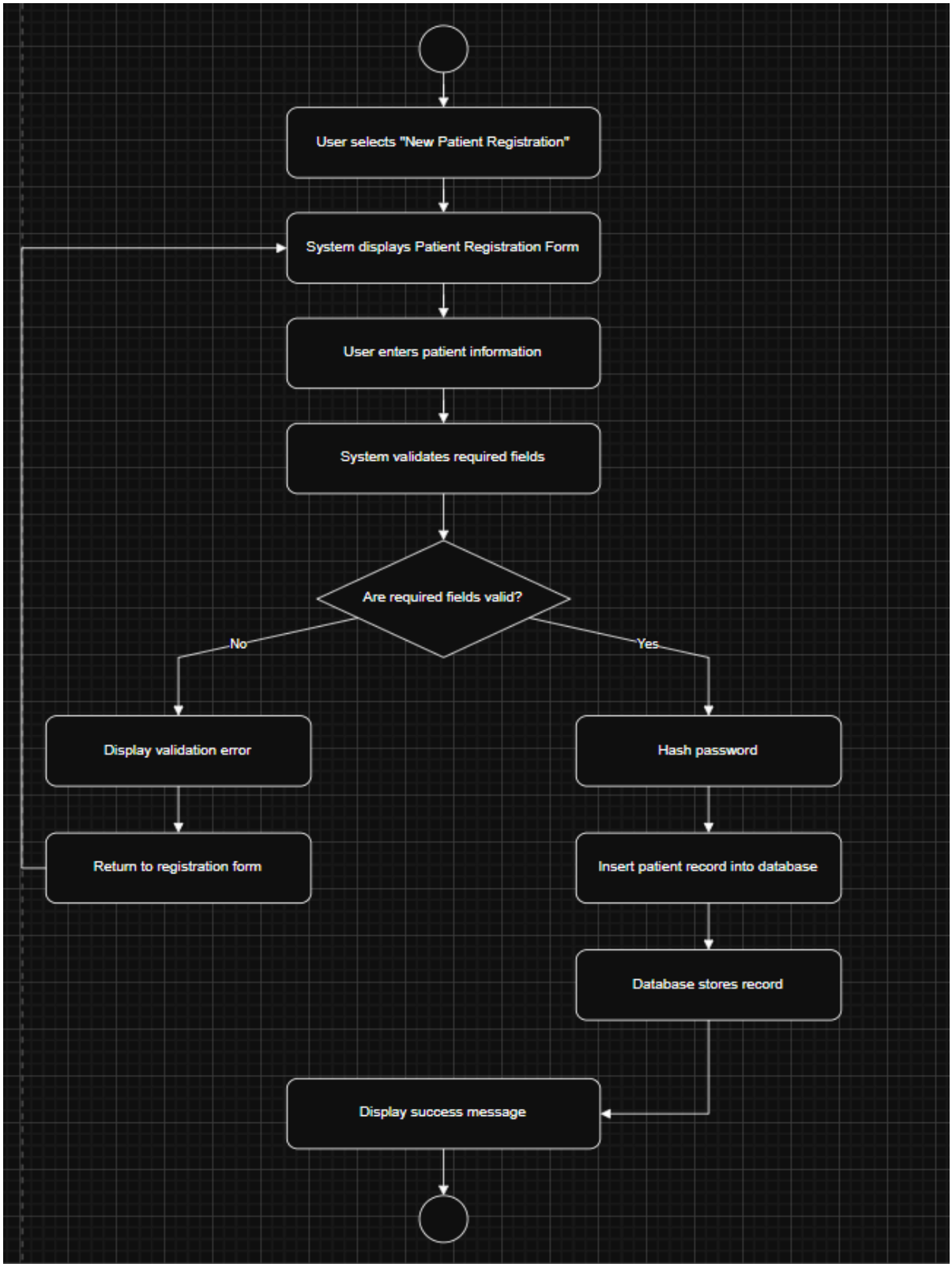
Logan Templin:(UPDATED)

System Sequence Diagram: (Add Patient)



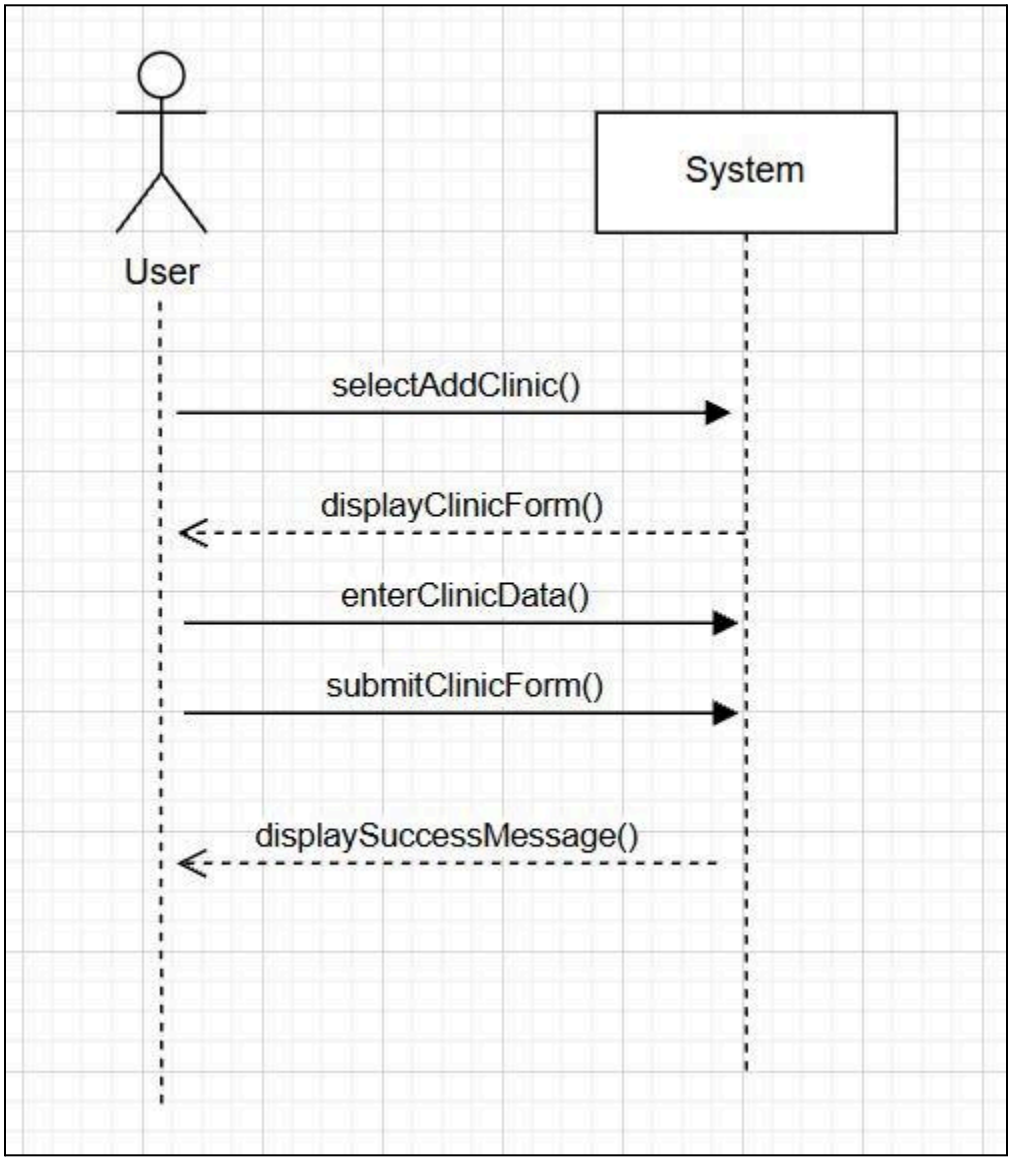
(Figure 11)

Activity Diagram (Add new Patient):

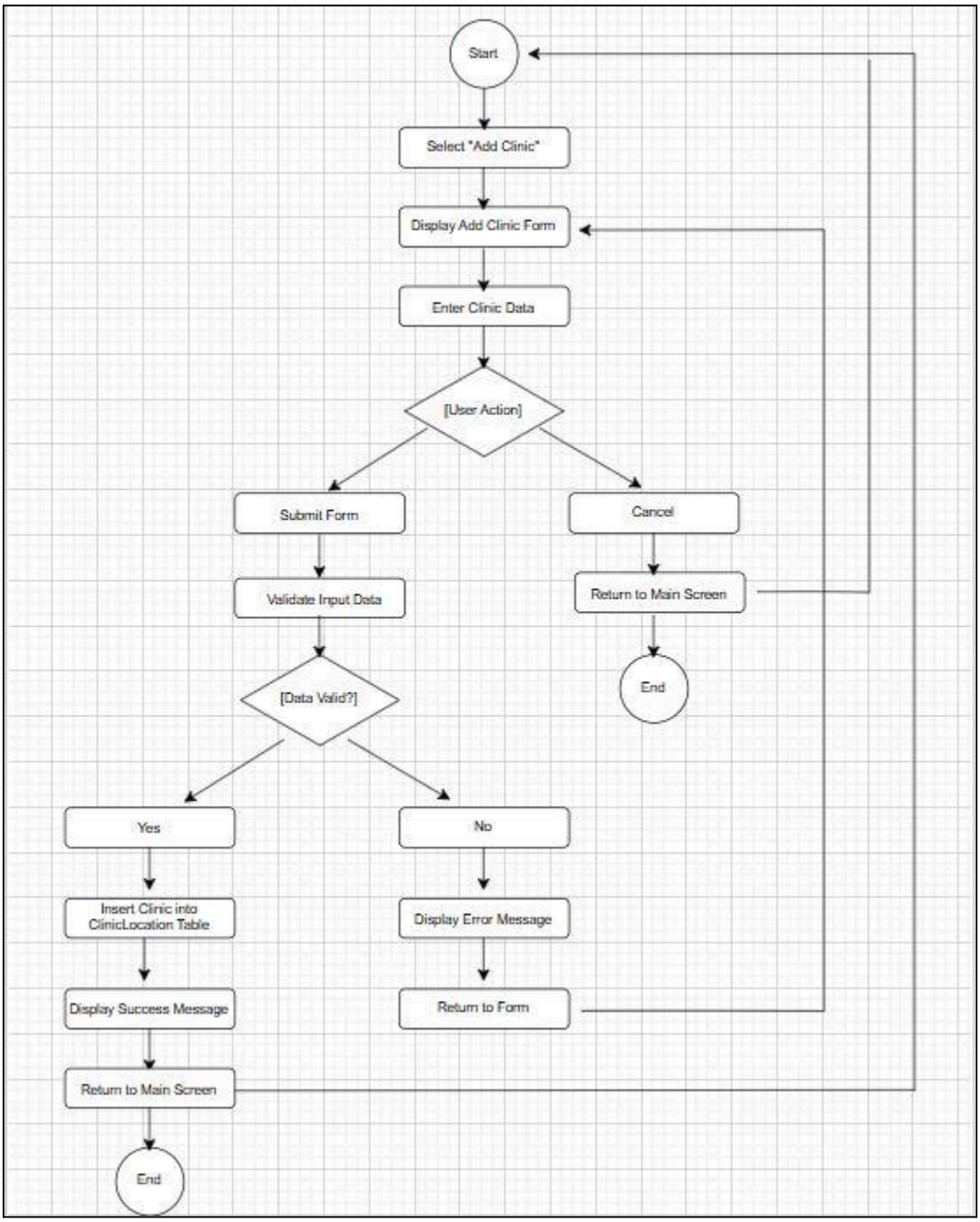


(Figure 12)

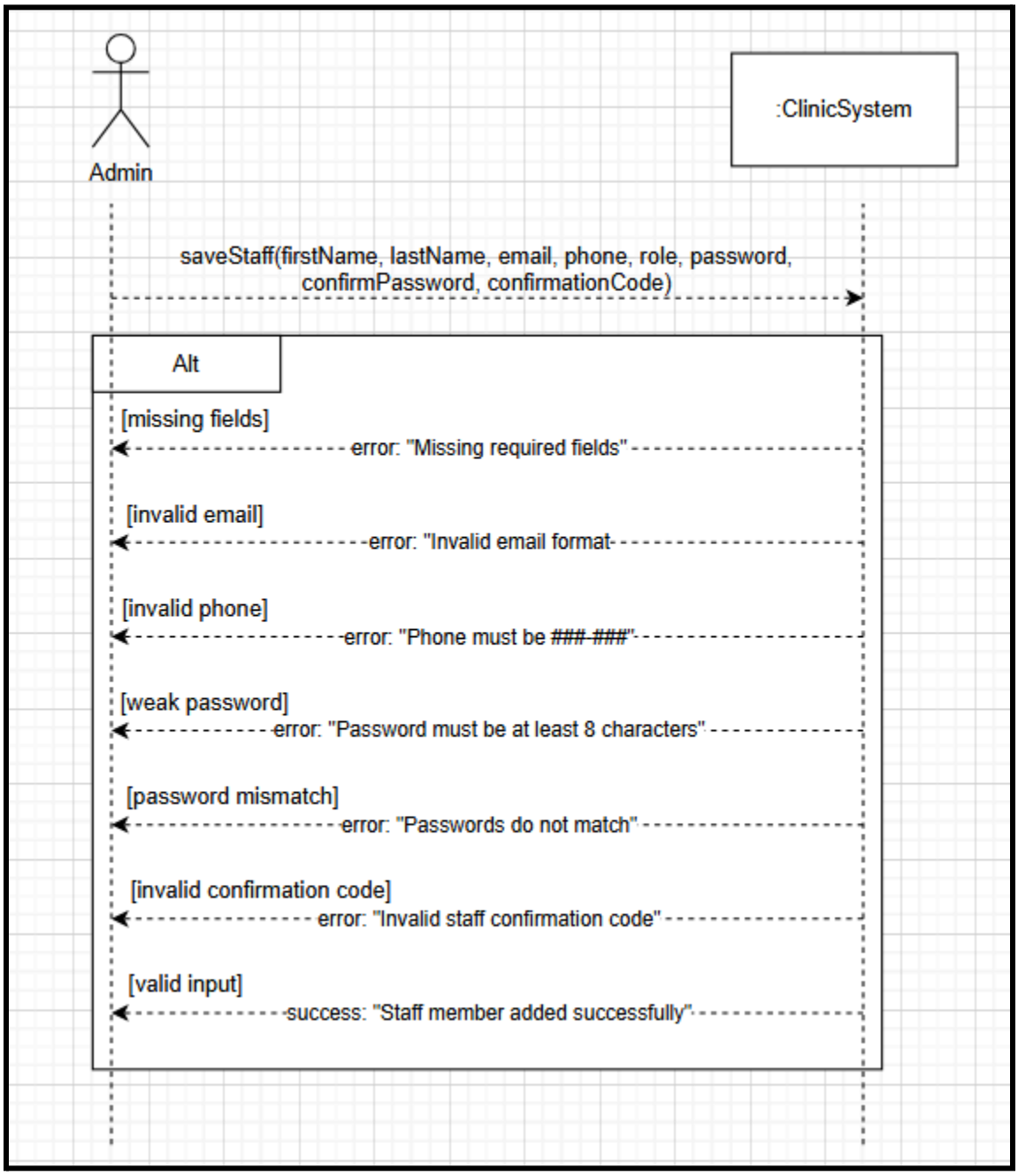
System Sequence Diagram: Add Clinic Location (updated) - Tessa Esman



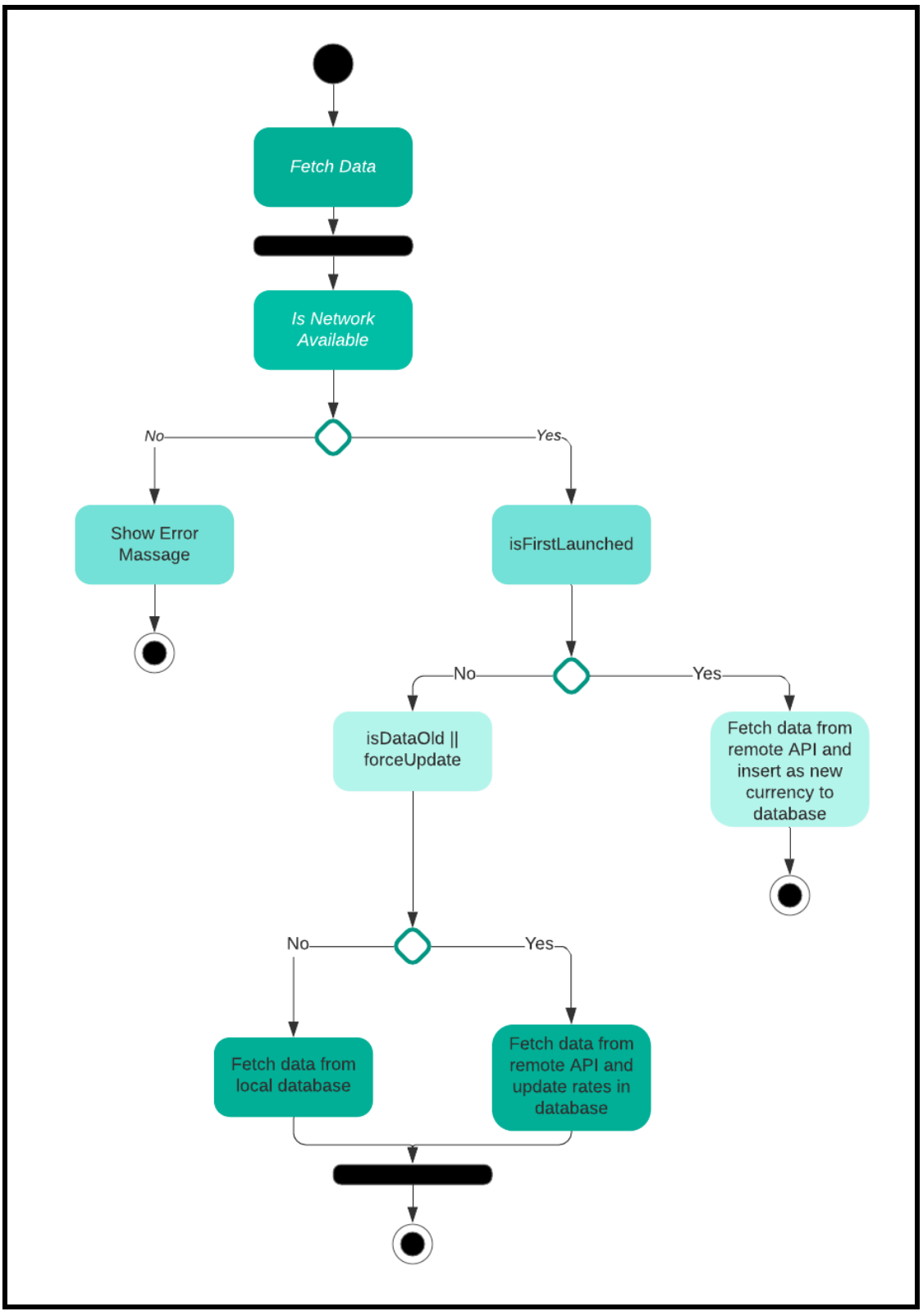
Activity Diagram: Add Clinic Location - Tessa Esman



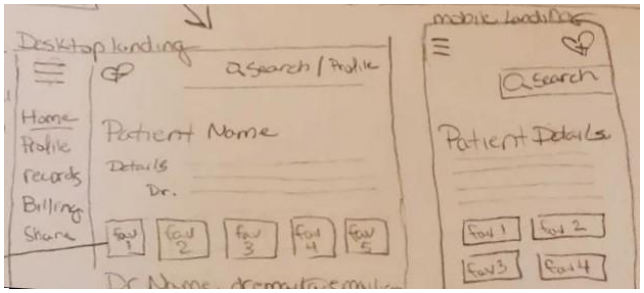
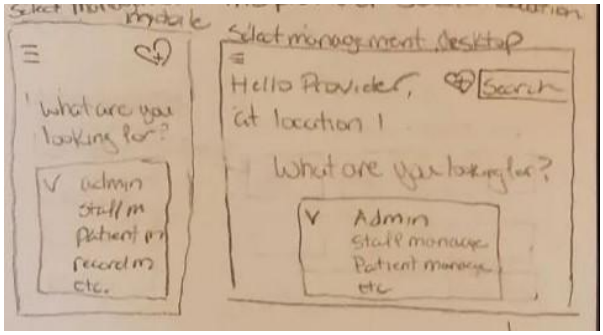
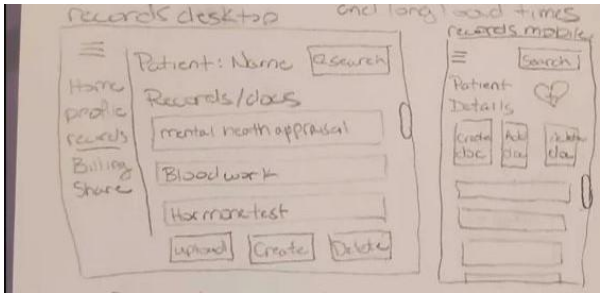
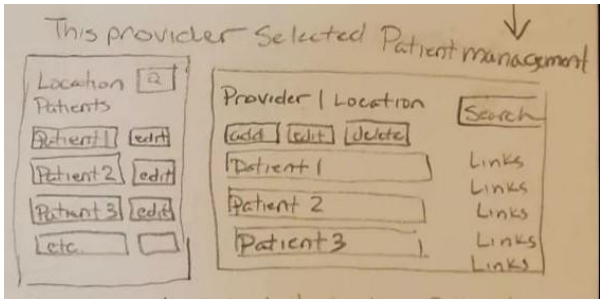
(Add New Staff) SSD: Mara Evans

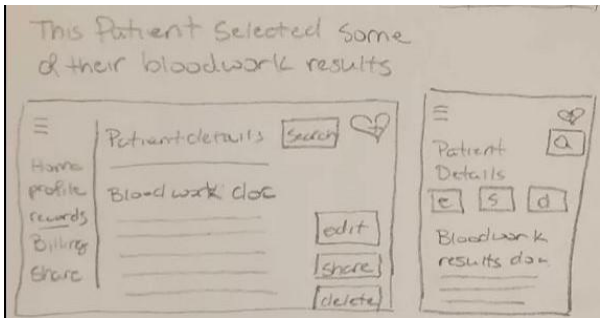


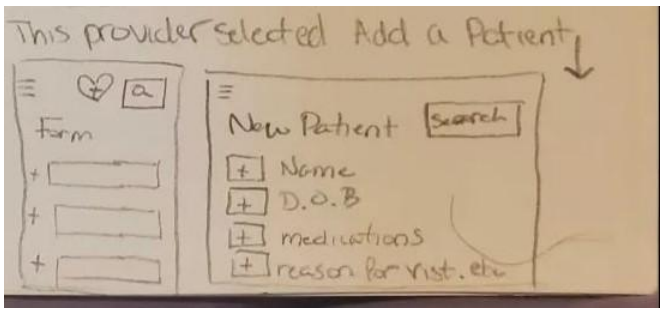
(Add New Staff) Activity Diagram: Mara Evans



User Interface Design
Old Wireframe:

Here the user has selected patient, and they are taken to the patients home screen, with all of their generic information, including saved favorites to cut down on choice overload and fatigue.	Here the user has selected provider, and they are taken to another selection screen where they decide which location they or the patient in question is located at for treatment. Then they are taken to another selection screen where they choose what use case they need.
	
Next the patient has selected the Records tab, and they are redirected to the records section where they can find all their documents, bloodwork, and test results.	This provider has selected patient management, and is redirected to the patient management menu, where they can see all of their current patients and links to assist in treatment.
	
Next this patient has selected some of their bloodwork results to share with a specific physician, and have editing options and Dr. signoff as well.	This provider has selected Add a patient, which allows them to create a new patient in the system and input their personal information to keep on record.

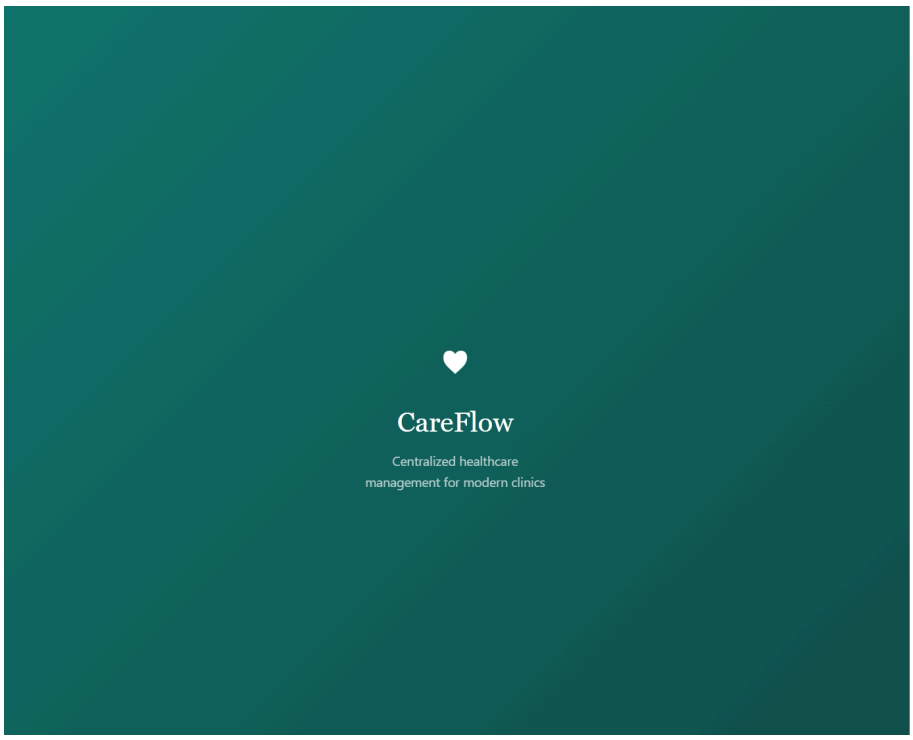


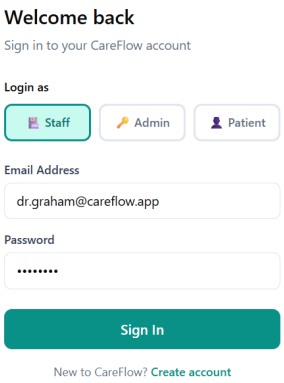


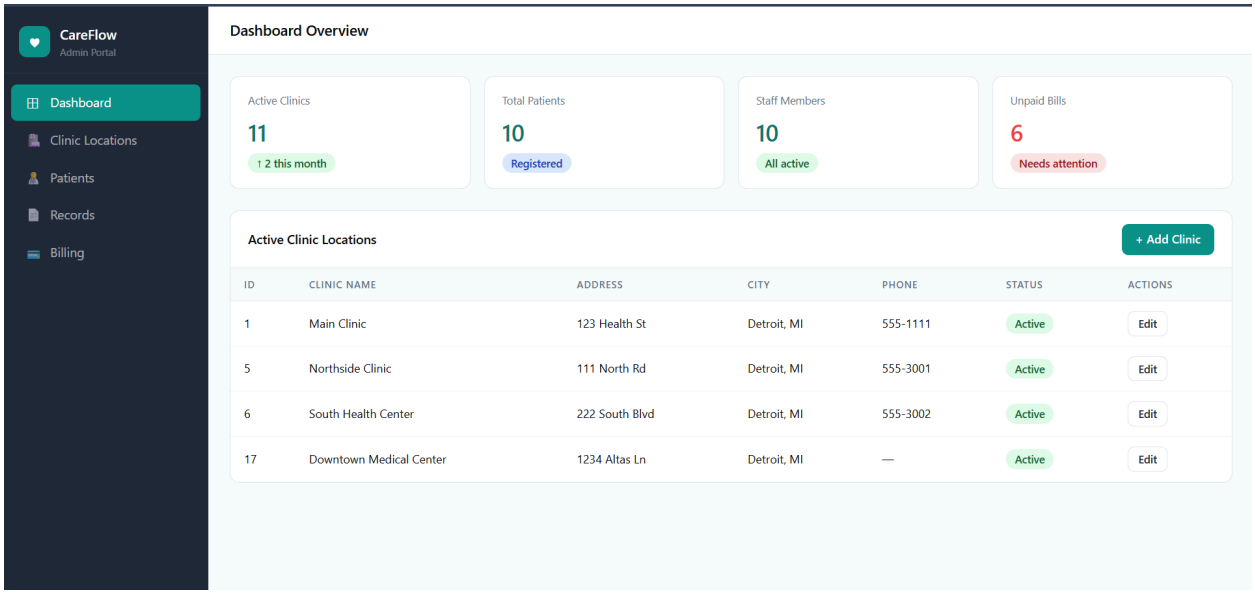
(Figure 17)

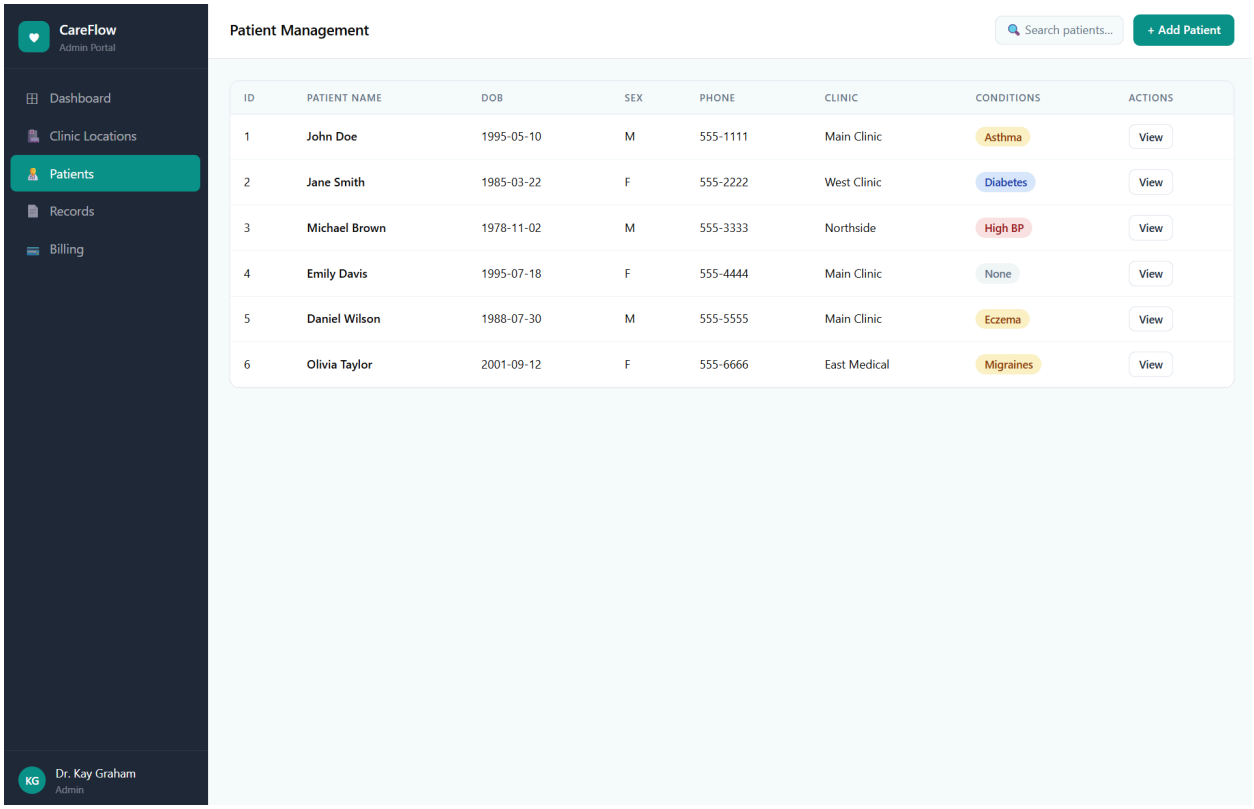
The wireframes included here cover the core functionalities and primary user workflows for both patient and provider roles. For the patient role, the wireframes illustrate dashboard access, profile management, viewing and updating patient records, and document sharing. For the provider role, the wireframes show dashboard access, clinic location selection, patient management, and adding new patients. Together, these wireframes represent the system's main use cases and demonstrate how each user role interacts with the platform to accomplish their tasks.

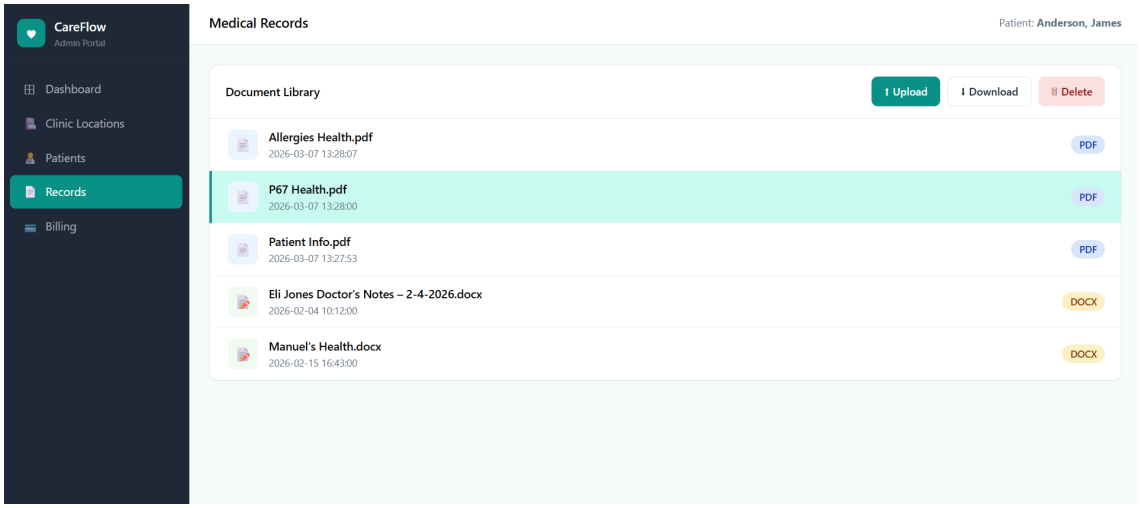
Professional UI mockups with data (FIGMA):

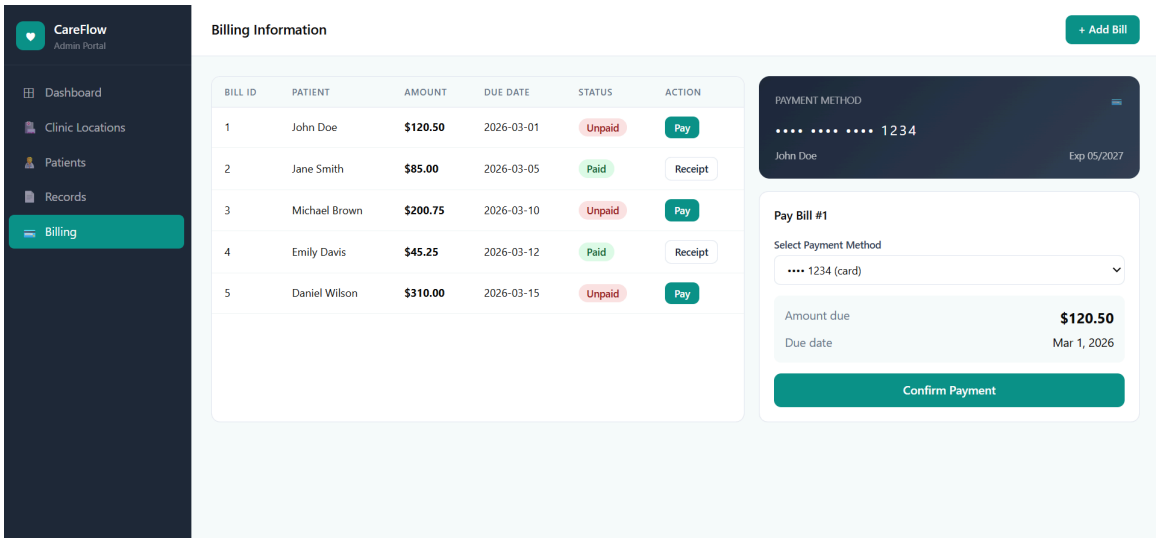


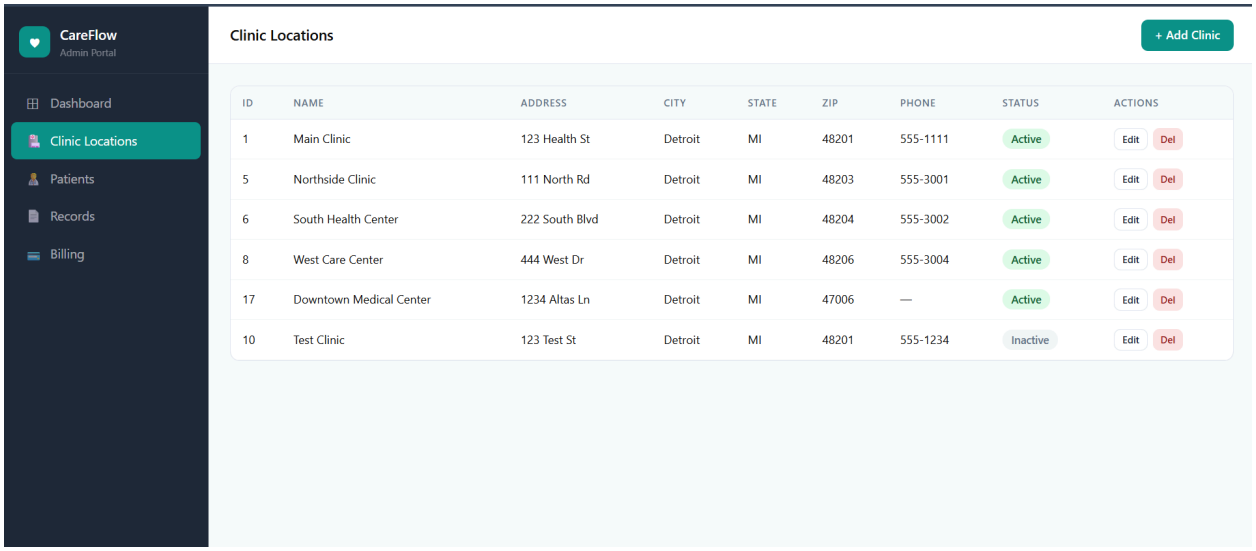



CareFlow
Admin Portal
Dashboard
Clinic Locations
Patients
Records
Billing
Dashboard Overview
Active Clinics
11
↑ 2 this month
Total Patients
10
Registered
Staff Members
10
All active
Unpaid Bills
6
Needs attention
Active Clinic Locations
+ Add Clinic
ID CLINIC NAME ADDRESS CITY PHONE STATUS ACTIONS
1 Main Clinic 123 Health St Detroit, MI 555-1111 Active Edit
5 Northside Clinic 111 North Rd Detroit, MI 555-3001 Active Edit
6 South Health Center 222 South Blvd Detroit, MI 555-3002 Active Edit
17 Downtown Medical Center 1234 Altas Ln Detroit, MI — Active Edit

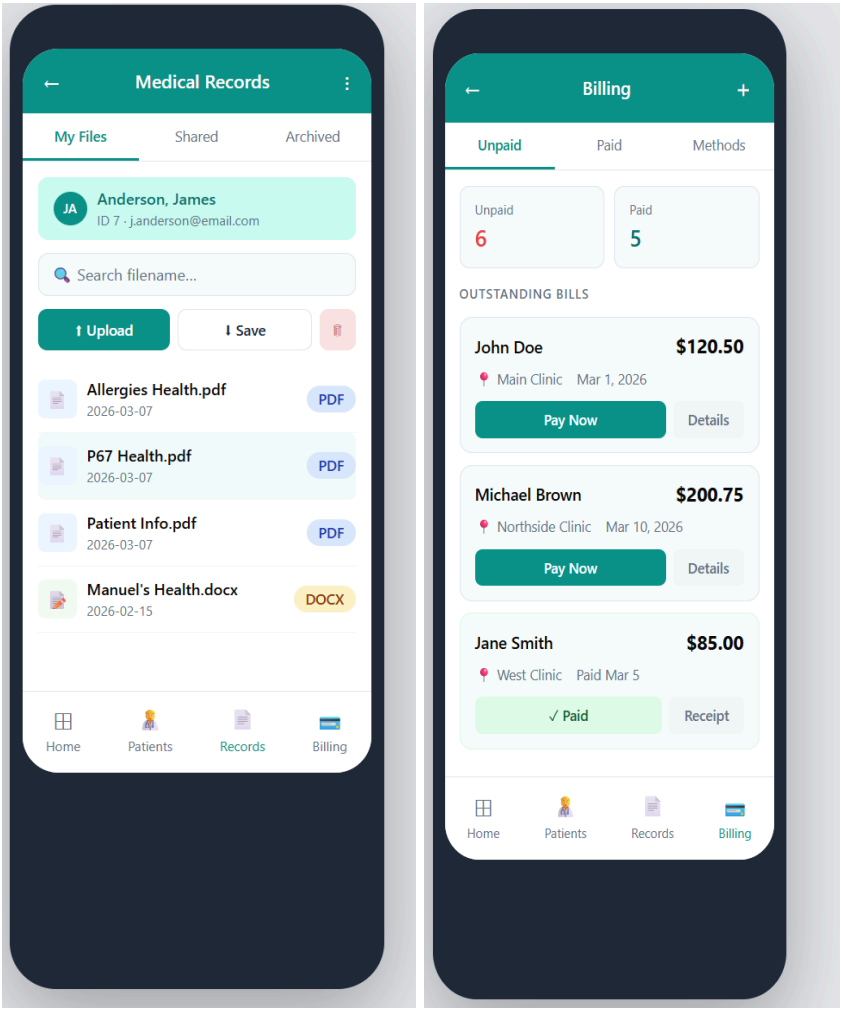

CareFlow
Admin Portal
Dashboard
Clinic Locations
Patients
Records
Billing
KG
Dr. Kay Graham
Admin
Patient Management
Search patients...
+ Add Patient
ID PATIENT NAME DOB SEX PHONE CLINIC CONDITIONS ACTIONS
1 John Doe 1995-05-10 M 555-1111 Main Clinic Asthma View
2 Jane Smith 1985-03-22 F 555-2222 West Clinic Diabetes View
3 Michael Brown 1978-11-02 M 555-3333 Northside High BP View
4 Emily Davis 1995-07-18 F 555-4444 Main Clinic None View
5 Daniel Wilson 1988-07-30 M 555-5555 Main Clinic Eczema View
6 Olivia Taylor 2001-09-12 F 555-6666 East Medical Migraines View


CareFlow
Admin Portal
Dashboard
Clinic Locations
Patients
Records
Billing
Medical Records
Patient: Anderson, James
Document Library
Upload
Download
Delete
Allergies Health.pdf
2026-03-07 13:28:07
PDF
P67 Health.pdf
2026-03-07 13:28:00
PDF
Patient Info.pdf
2026-03-07 13:27:53
PDF
Eli Jones Doctor's Notes – 2-4-2026.docx
2026-02-04 10:12:00
DOCX
Manuel's Health.docx
2026-02-15 16:43:00
DOCX


CareFlow
Admin Portal
Dashboard
Clinic Locations
Patients
Records
Billing
Billing Information
+ Add Bill
BILL ID
PATIENT
AMOUNT
DUE DATE
STATUS
ACTION
1 John Doe $120.50 2026-03-01 Unpaid Pay
2 Jane Smith $85.00 2026-03-05 Paid Receipt
3 Michael Brown $200.75 2026-03-10 Unpaid Pay
4 Emily Davis $45.25 2026-03-12 Paid Receipt
5 Daniel Wilson $310.00 2026-03-15 Unpaid Pay
PAYMENT METHOD
•••• •••• •••• 1234
John Doe
Exp 05/2027
Pay Bill #1
Select Payment Method
•••• 1234 (card)
Amount due $120.50
Due date Mar 1, 2026
Confirm Payment

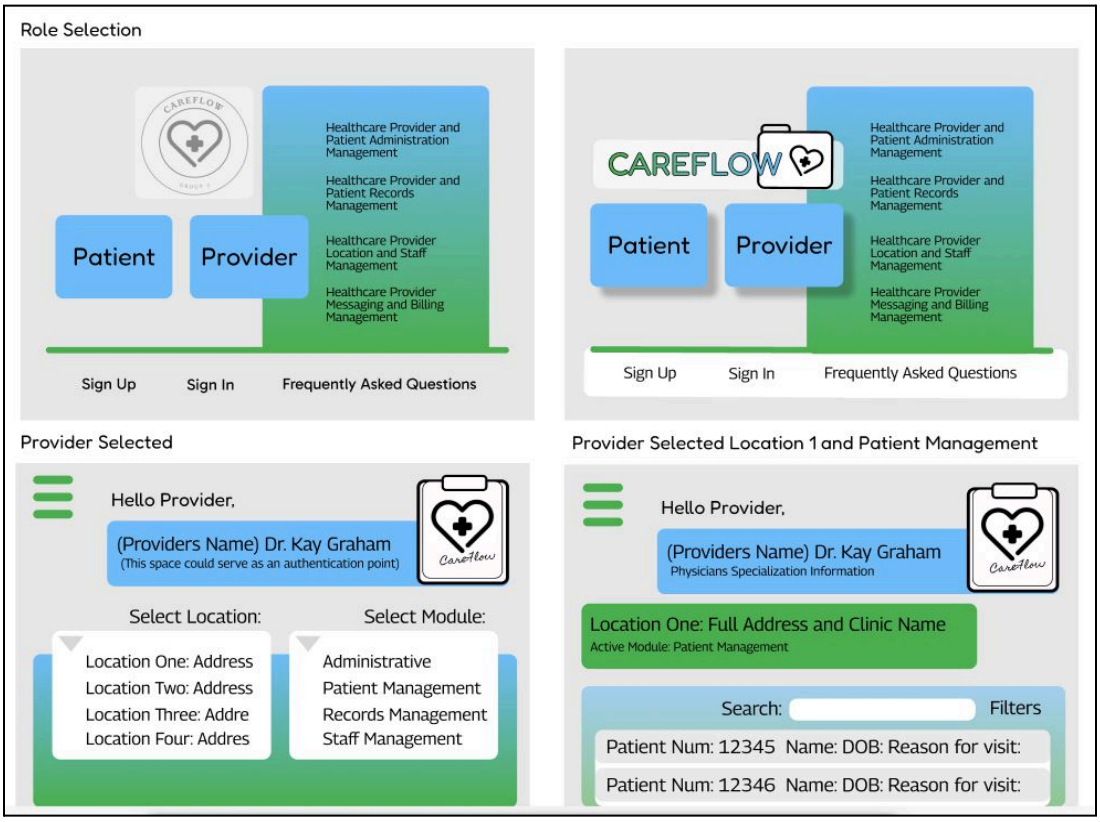

CareFlow
Admin Portal
Dashboard
Clinic Locations
Patients
Records
Billing
Clinic Locations
+ Add Clinic
ID
NAME
ADDRESS
CITY
STATE
ZIP
PHONE
STATUS
ACTIONS
1 Main Clinic 123 Health St Detroit MI 48201 555-1111 Active Edit Del
5 Northside Clinic 111 North Rd Detroit MI 48203 555-3001 Active Edit Del
6 South Health Center 222 South Blvd Detroit MI 48204 555-3002 Active Edit Del
8 West Care Center 444 West Dr Detroit MI 48206 555-3004 Active Edit Del
17 Downtown Medical Center 1234 Altas Ln Detroit MI 47006 — Active Edit Del
10 Test Clinic 123 Test St Detroit MI 48201 555-1234 Inactive Edit Del

Responsive:

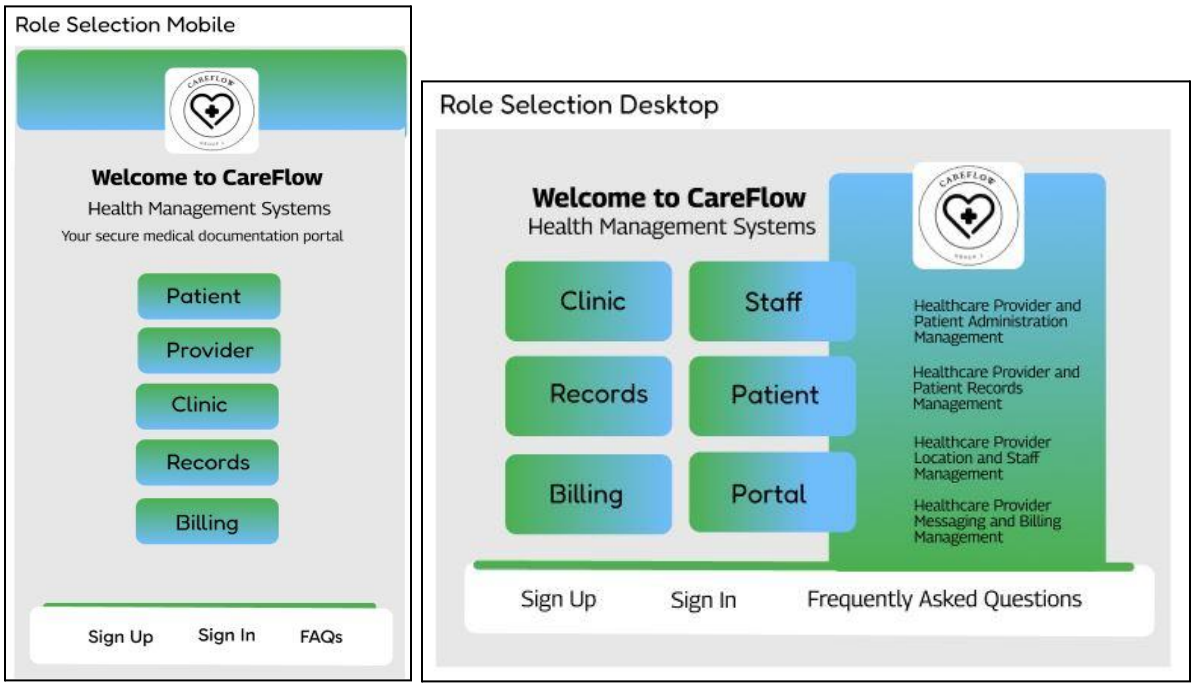


Both desktop and mobile versions are included for each wireframe, ensuring that the design is responsive and functional across different screen sizes. Desktop layouts utilize tables, panels, and sidebars for efficient data presentation, while mobile layouts adopt vertical stacks, cards, and hamburger menus to accommodate smaller screens. This approach satisfies the responsive design requirement and illustrates how key actions and workflows remain accessible and usable regardless of device.

Old/Alternative UI Mockups with Data:



(Figure 18) Original



(Figure 19) Revised / Responsive Design

HCI Rules Discussion:

Rule 5: Prevent Errors

Definition:

The interface should be designed so users cannot make serious errors and when errors do occur, the system should provide simple and specific instructions for recovery.

Application in CareFlow:

- Implemented on the Patient Billing.
- When adding a staff registration:
  - A Drop down list is provided to prevent user error in selecting a role as well as a confirmation code for certain role

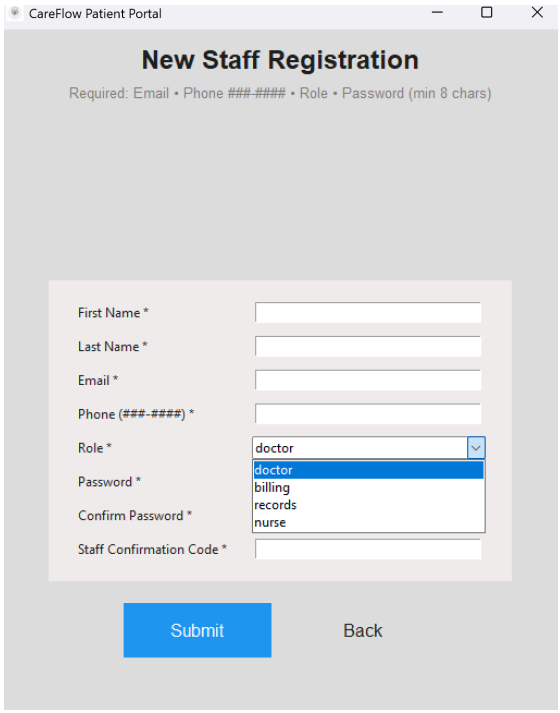


Figure 20

Rule 3: Offer Informative feedback

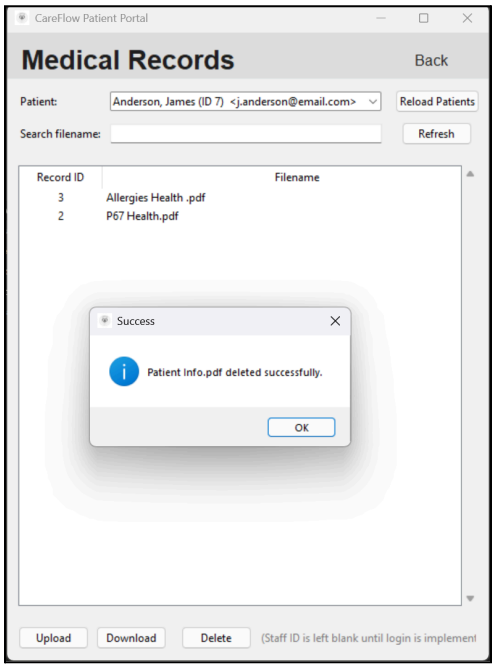
Meaning:

Every user action should produce interface feedback with more substantial responses for major or infrequent actions.

Application in CareFlow:

CareFlow applies this rule consistently across the Clinic Locations and Medical Records pages. On the Medical Records page, uploading a file shows “[filename] uploaded successfully." and

deleting a record shows “[filename] deleted successfully.” Every path, including failures produces a messagebox response so the user always knows the outcome of their actions. As shown in figure 20.



(Figure 20)

Rule 6: Permit Easy Reversal of Actions
Meaning:
Actions should be reversible wherever possible, which relieves user anxiety and encourages exploration.

Application in CareFlow:
CareFlow applies this rule on the Clinic Locations page through a soft delete mechanism. When a clinic is deleted, the soft_delete_clinic_location() function sets status = 'inactive' in the database rather than removing the row entirely. The clinic disappears from the active table because get_all_active_clinics() only fetches rows where status = 'active', but the data is preserved and can be restored by updating the status back. This means a mistakenly deleted clinic is recoverable without any data loss. Additionally, both the Add and Update dialogs include a confirmation step before committing — on the delete action specifically, a messagebox.askyesno("Confirm Delete", ...) prompt gives the user a clear exit before the action takes effect.

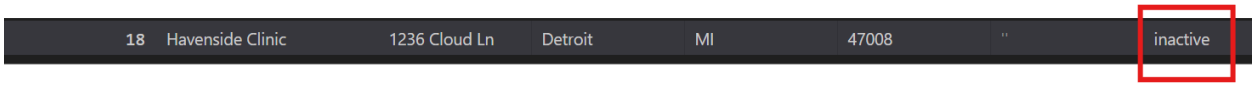
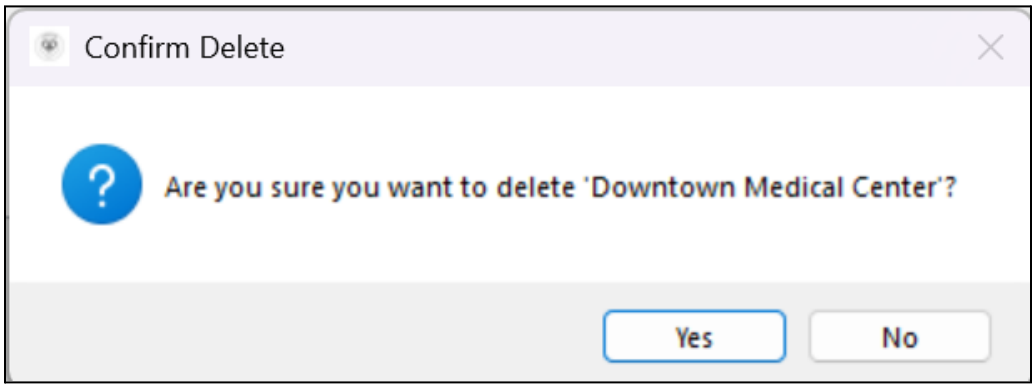


Figure xx, inactive in Clinic location database



(Figure 21)

Final Screenshots:

Include final screenshots of the implemented features and explain the sequence of events (storyboard).

Use Case 1: Upload

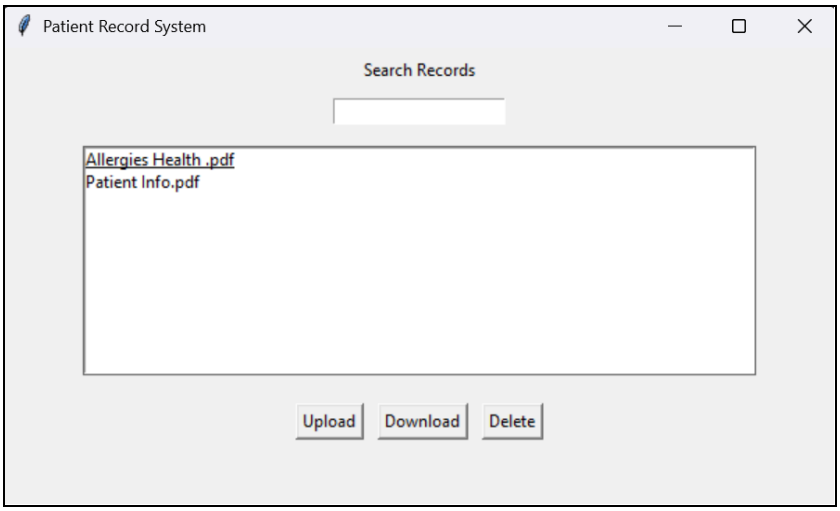
Description:

This use case allows the user to upload a medical file into the system. The selected file is copied from its original location into the system's designated storage folder. Once copied, the file list updates automatically to display the newly uploaded record.

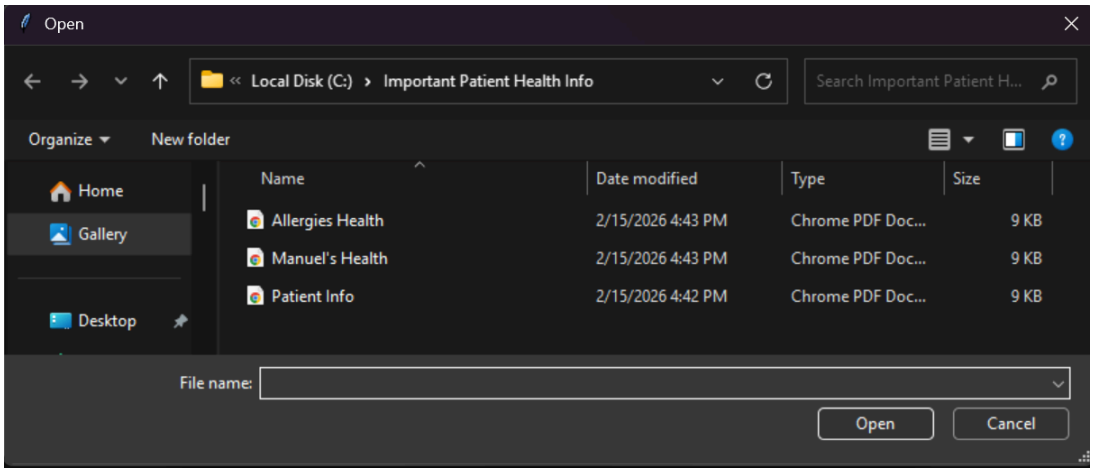
Process:

1. User clicks the Upload button.
2. A file selection dialog appears.
3. User selects a file.
4. The file is copied into the record_files directory.
5. The system refreshes the file list.
6. A success message is displayed.

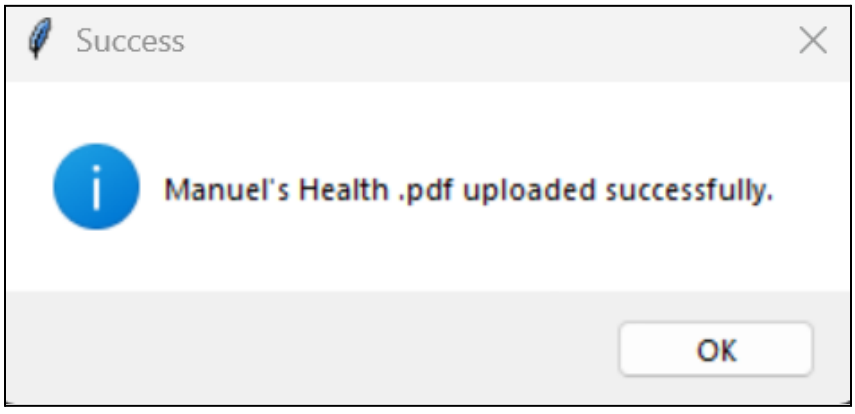
User Clicks Uploads



File Directory is Opened



Upload Success message



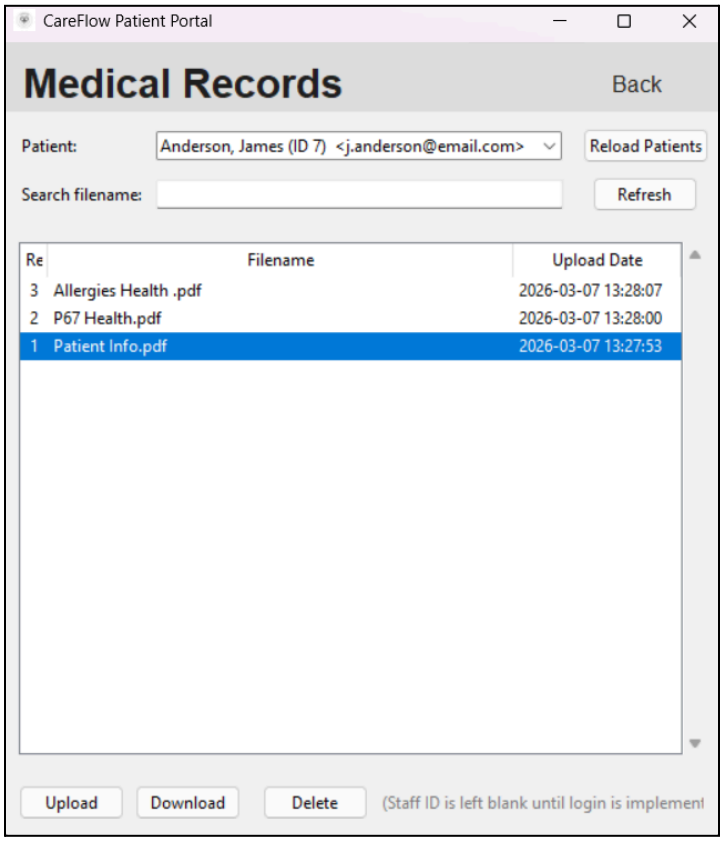
Use Case 4: Download

Description:
This use case allows the user to retrieve a stored patient file from the system and save it to a selected location on their device.

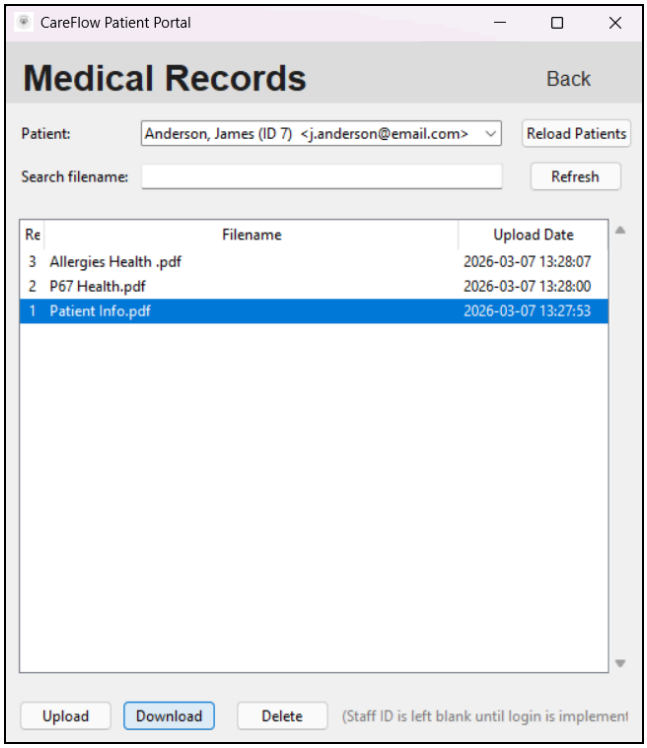
Process:

1. User selects a file from the list.
2. User clicks the Download button.
3. A save location dialog appears.
4. User selects destination folder.
5. The file is copied to the selected location.

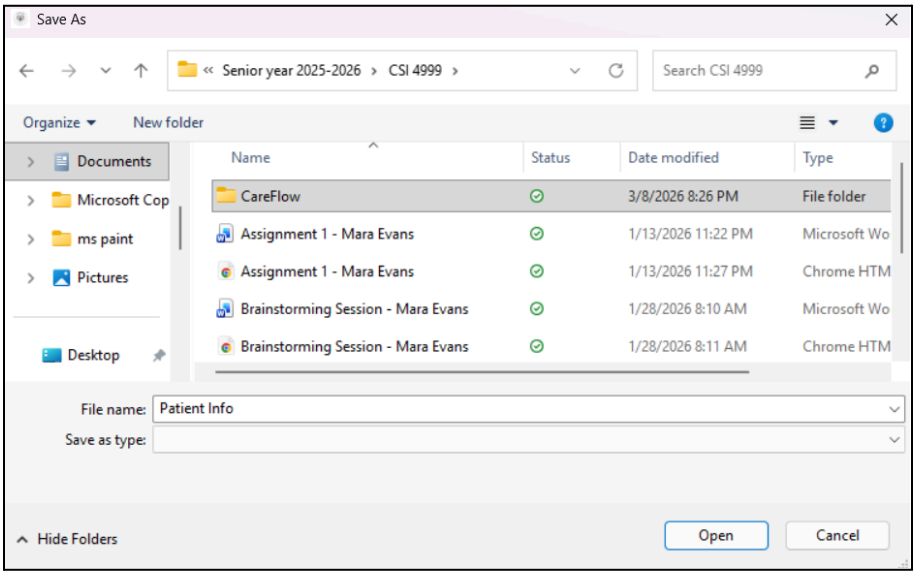
User selects a file from the list:



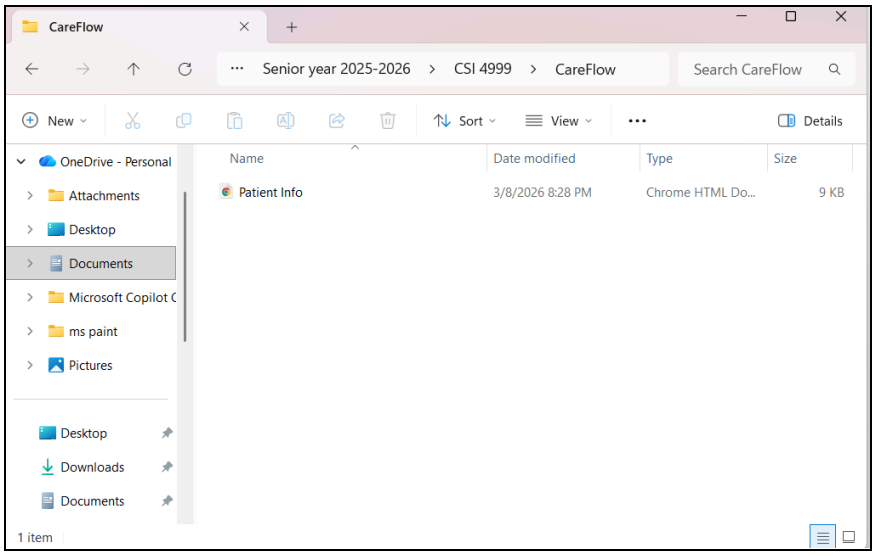
Use clicks the Download button:



User selects a folder to save the file to:



File is saved in the indicated folder:



Use Case 1: Search Files

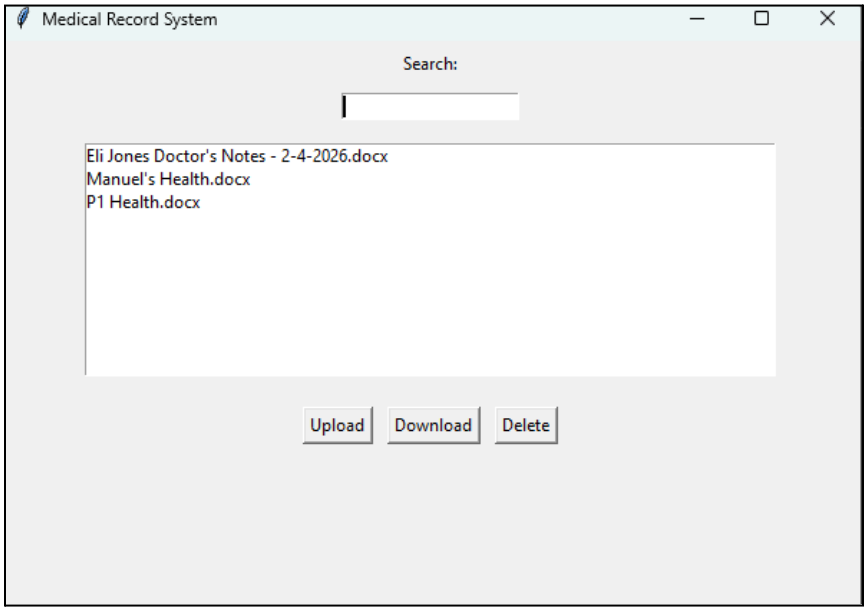
Description:

This use case allows users to use a search bar to search for a specific record file within the records table. When the user types the name, the listbox will refresh, filtering out filenames not relevant to what they are typing. The listbox will update live as they type.

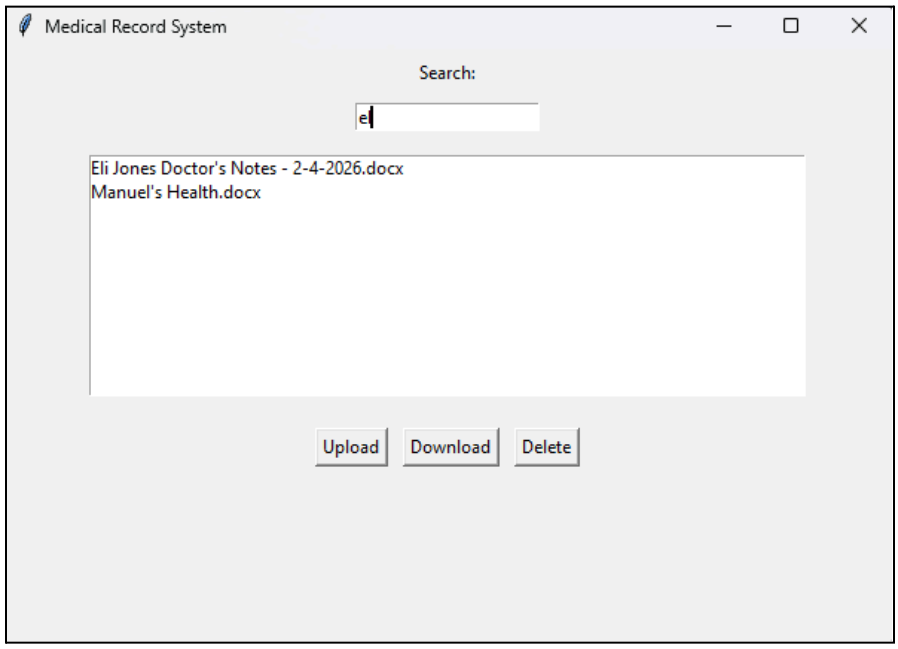
Process:

1. The user click on the search box above the listbox
2. The user types the filename or terms relevant to the filename
3. The listbox updates live to match the user's input

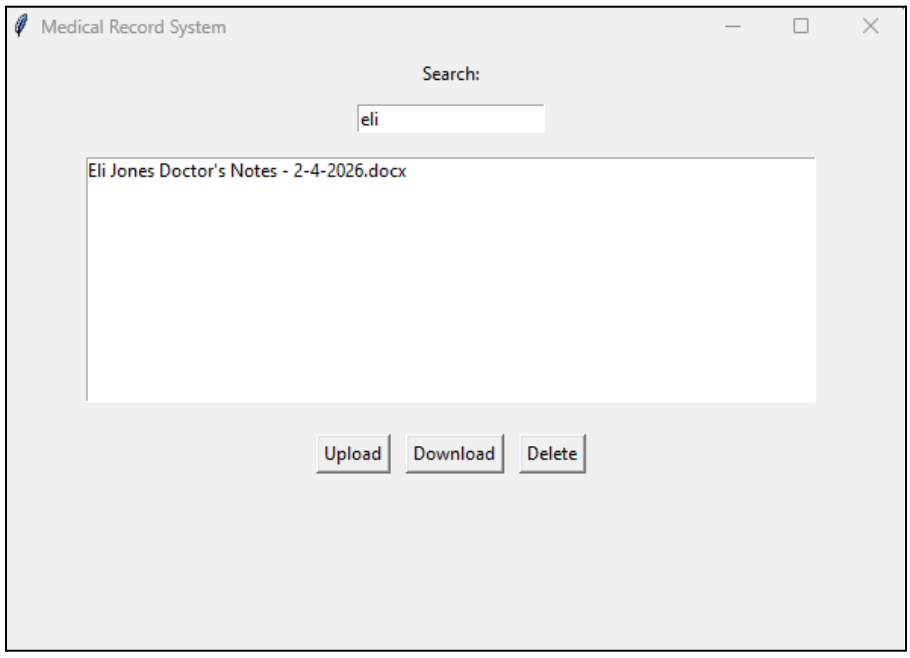
User Clicks on Searchbox:



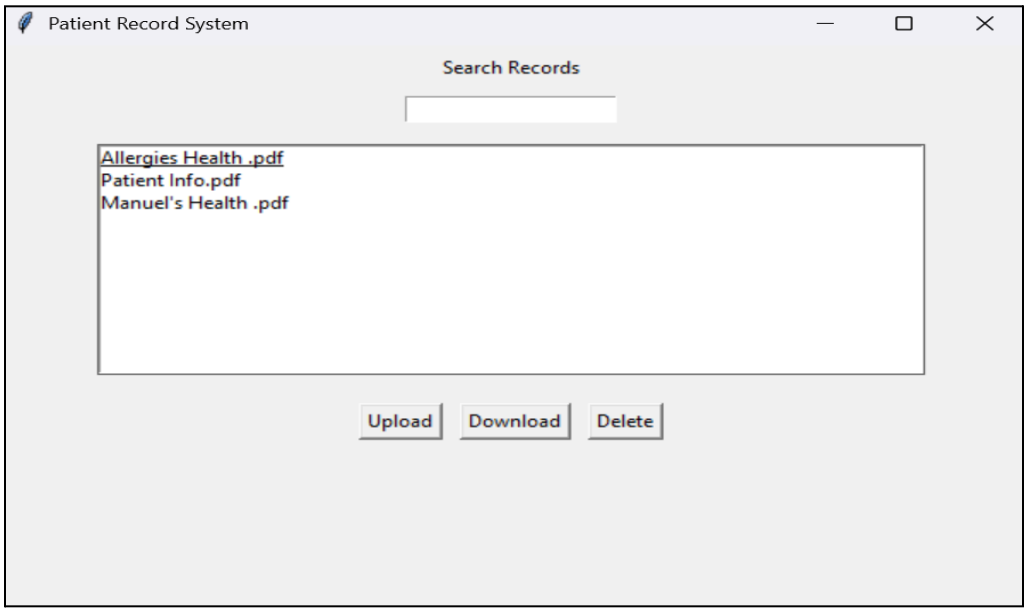
User Types Final Name and Listbox Filters



Listbox Updates to Match User's Input



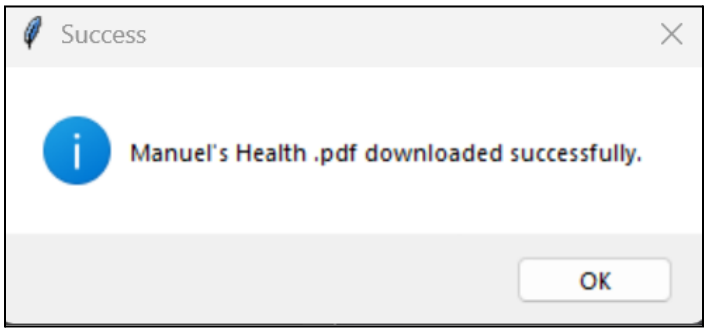
User selects Download



File Directory Opens: User selects save



Download Success Message



Use Case 3: Delete

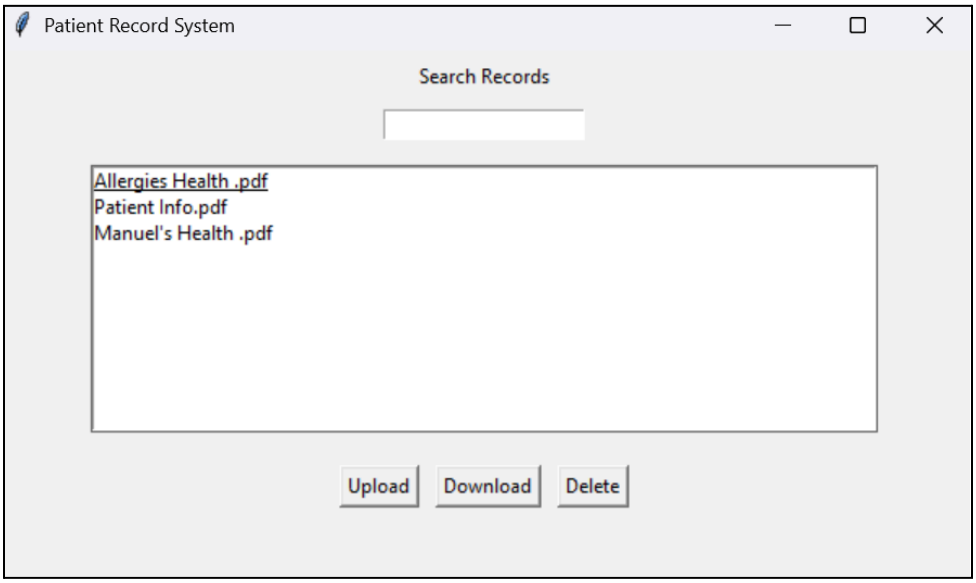
Description:

This use case allows the user to permanently remove a patient file from the system storage.

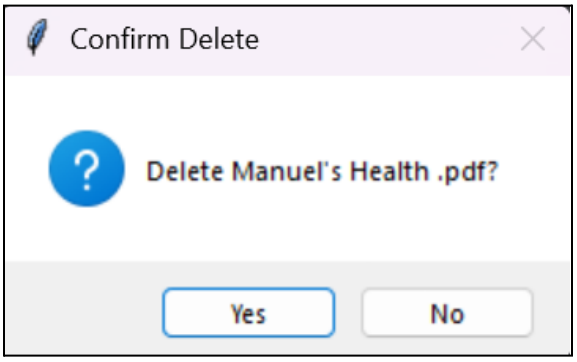
Process:

1. User selects a file.
2. User clicks the Delete button.
3. System asks for confirmation.
4. If confirmed, the file is removed from the storage folder.
5. The file list updates automatically.

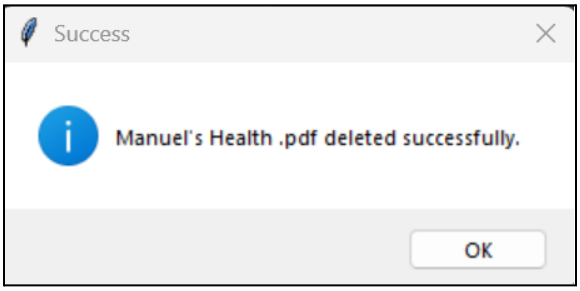
User Selects Delete



Confirmation Message



Success Message



Use Case 5: Add new clinic location - Tessa Esman

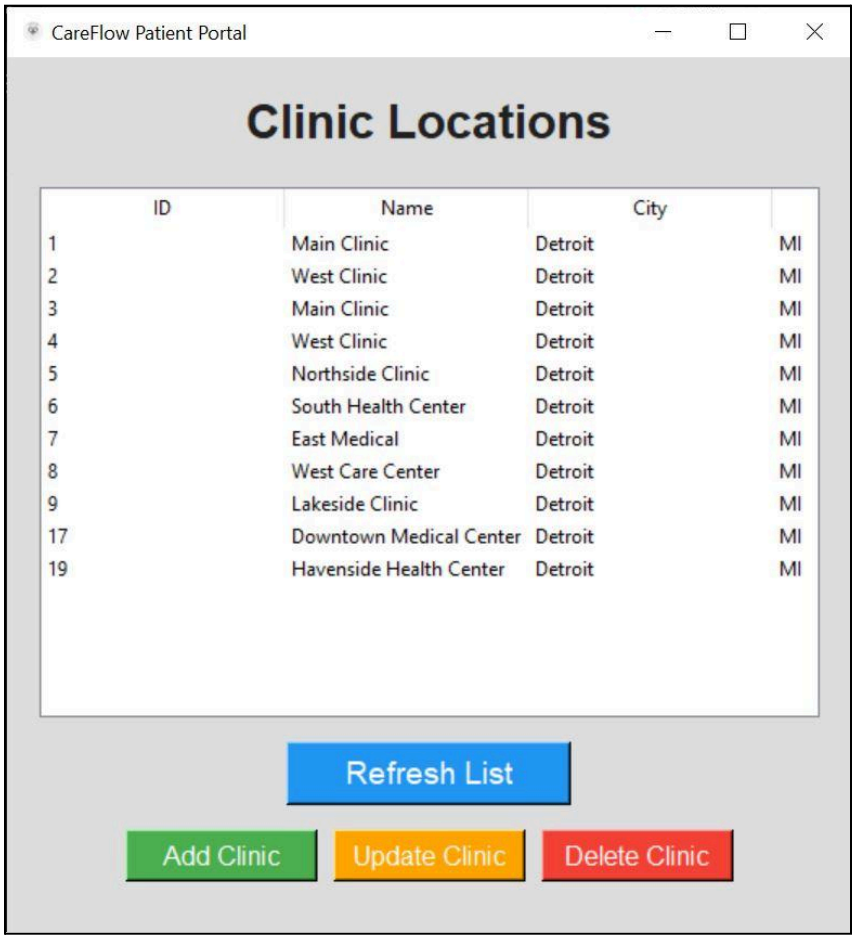
Description:

This use case allows authorized users to create and store a new clinic location record in the system. This ensures that new clinic locations can be tracked, managed, and accessed within the application.

Process:

1. User selects the Add clinic button
2. The system prompts user to enter required clinic details (clinic name, address, phone number, services, etc.)
3. User inputs the required information
4. System validates the input
5. System saves the new clinic record to the database
6. System displays a confirmation message indicating the clinic was added successfully

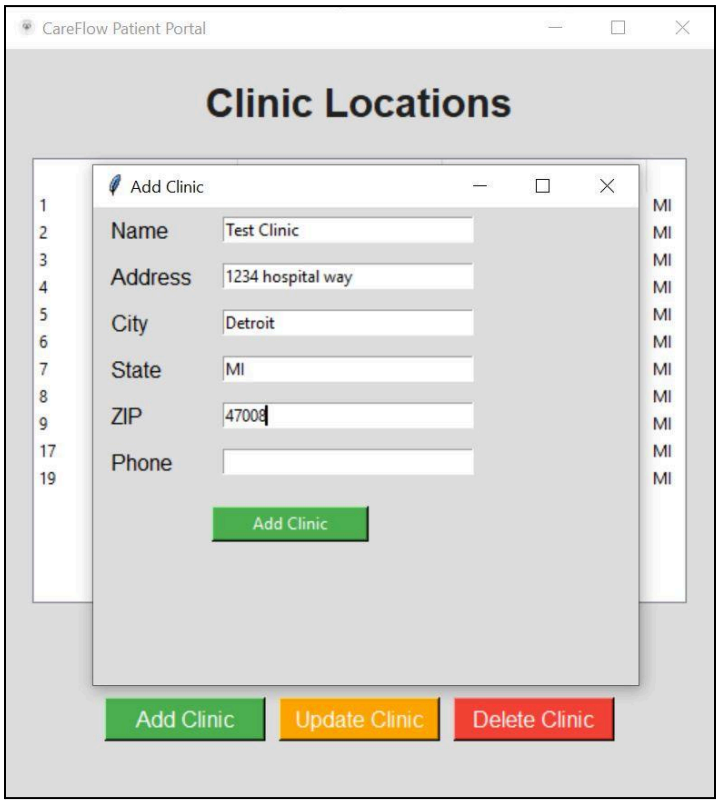
Add clinic button click



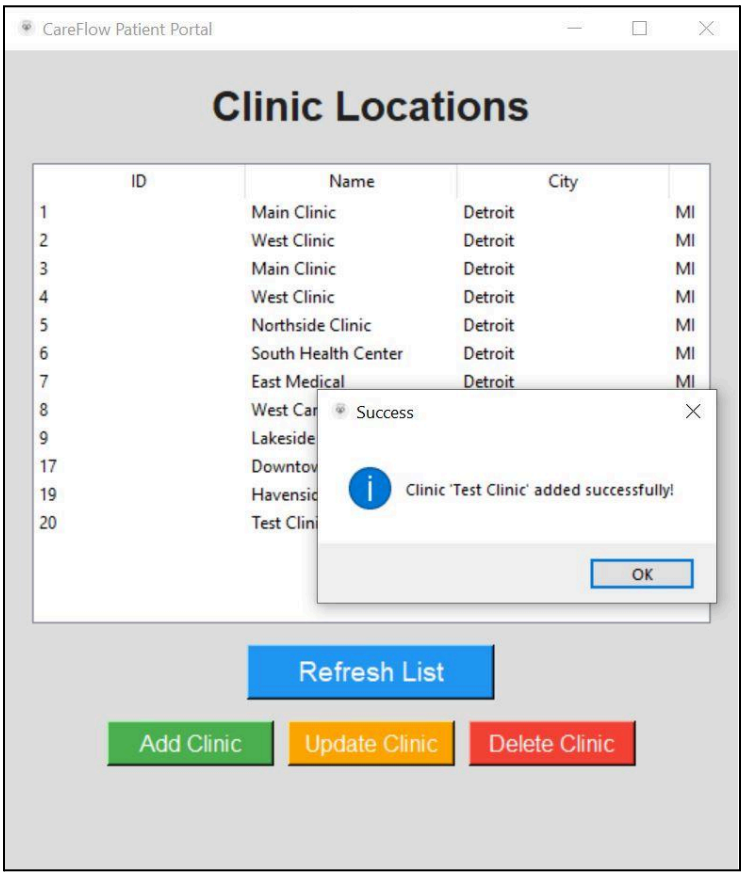
User prompt



User input



Confirmation message



Use Case 6: Update Clinic Location - Tessa Esman

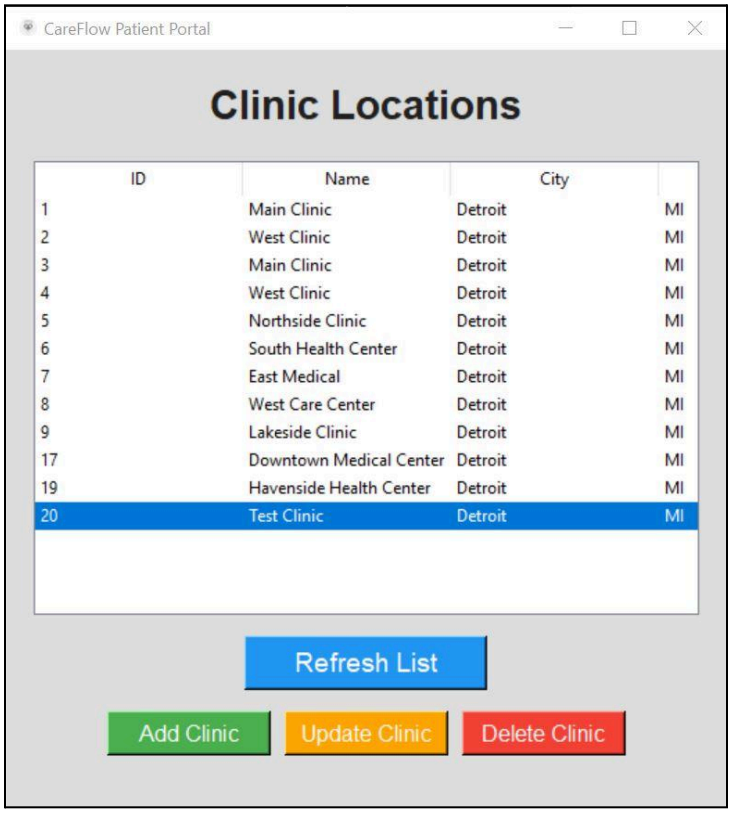
Description:

The Update clinic use case allows an authorized user to modify existing clinic information. This ensures that clinic records remain accurate and up to date.

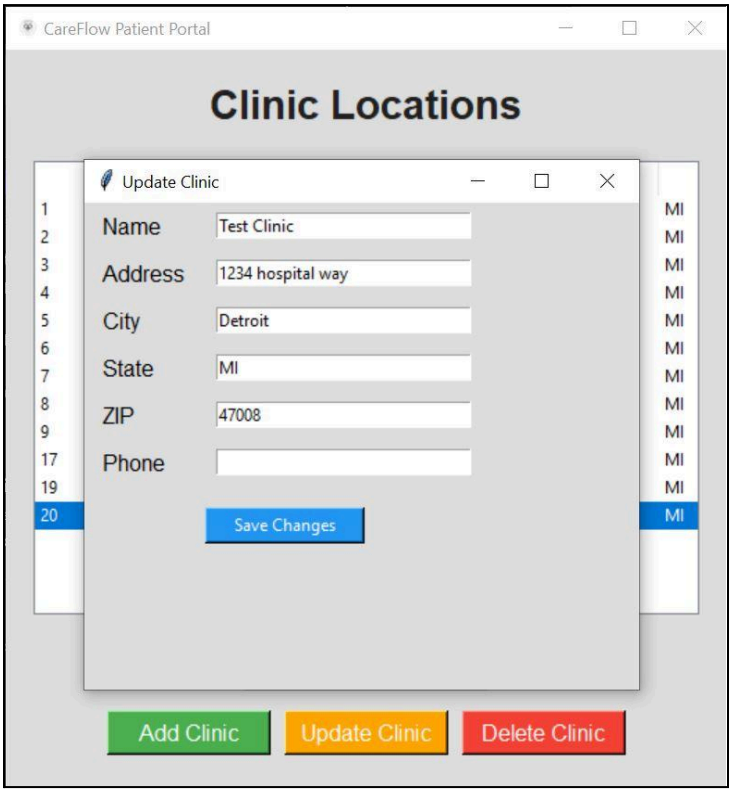
Process:

1. System displays a list of existing clinics
2. User selects the clinic to update by clicking listing
3. System displays the current clinic details
4. User enters updated information for one or more fields
5. System validates new data
6. System saves the updated information to the database
7. System displays a confirmation message indicating the update was successful
8. System updates displayed list

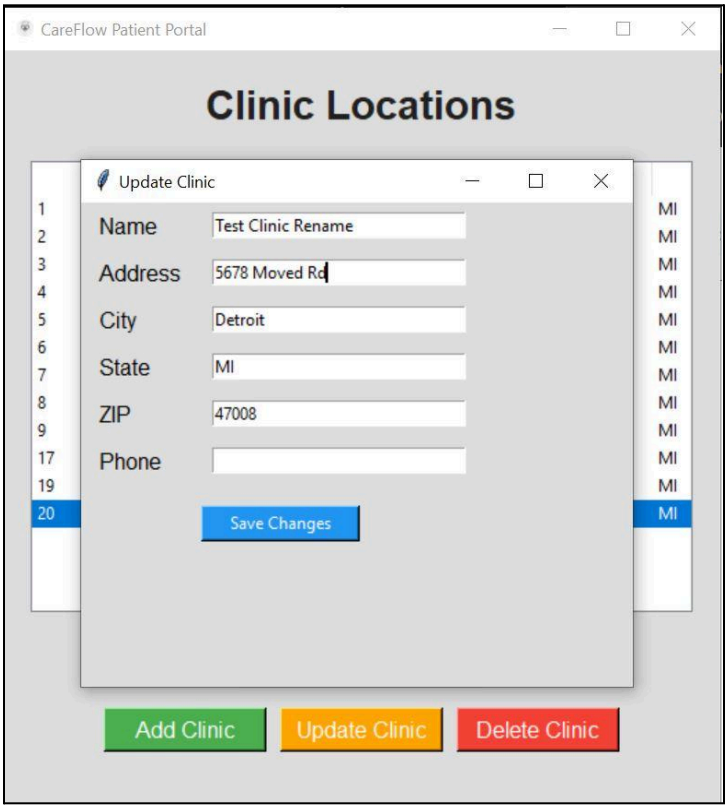
Select Clinic to update, then click update button



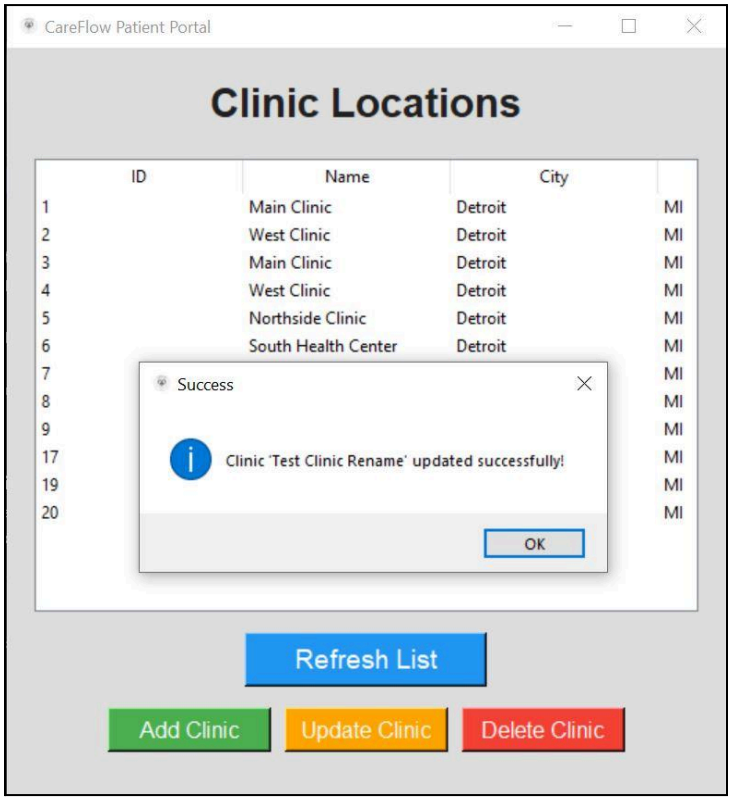
Current details displayed



User updates information



Confirmation message



Updated listings

8	West Care Center	Detroit	MI
9	Lakeside Clinic	Detroit	MI
17	Downtown Medical Center	Detroit	MI
19	Havenside Health Center	Detroit	MI
20	Test Clinic Rename	Detroit	MI

Use Case 7: Delete Clinic Location - Tessa Esman

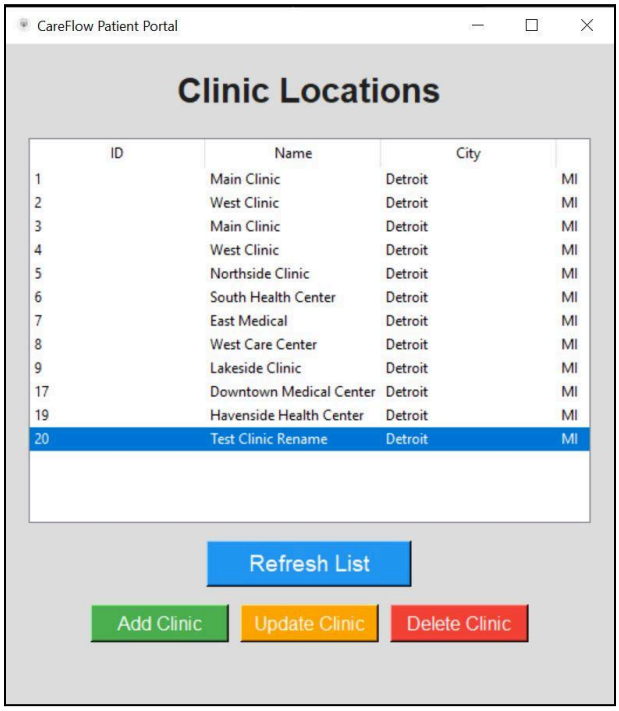
Description:

The delete clinic location use case allows an authorized user to soft delete/ remove a clinic record from the system when it is no longer needed.

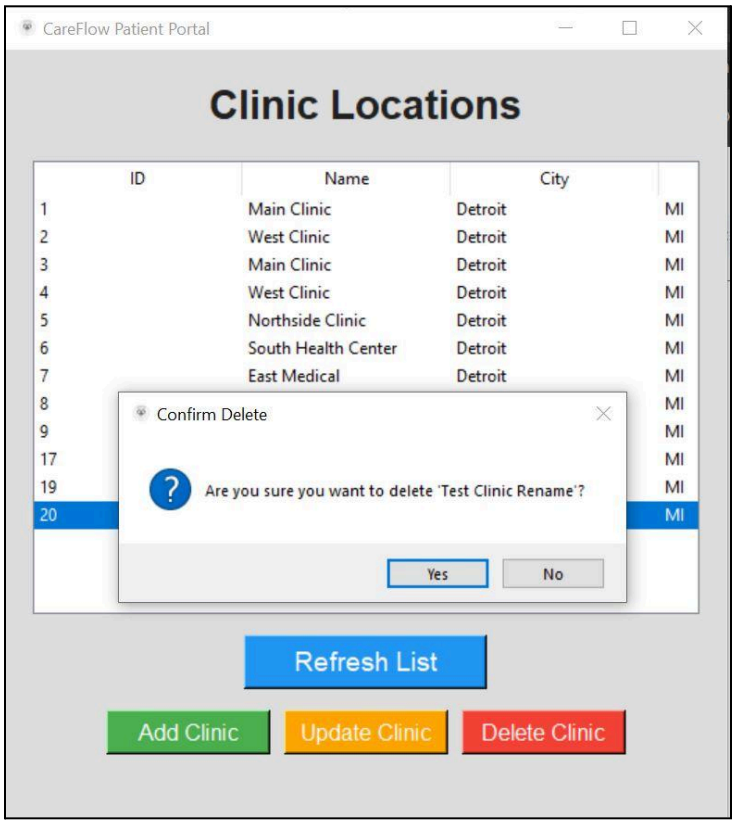
Process:

1. User selects clinic location entry they wish to delete from list
2. User clicks the delete clinic button
3. System prompts the user to confirm the deletion
4. User confirms the action
5. System removes the clinic record from the database
6. System displays a confirmation message indicating the clinic was successfully deleted
7. Displays updated list post location removal

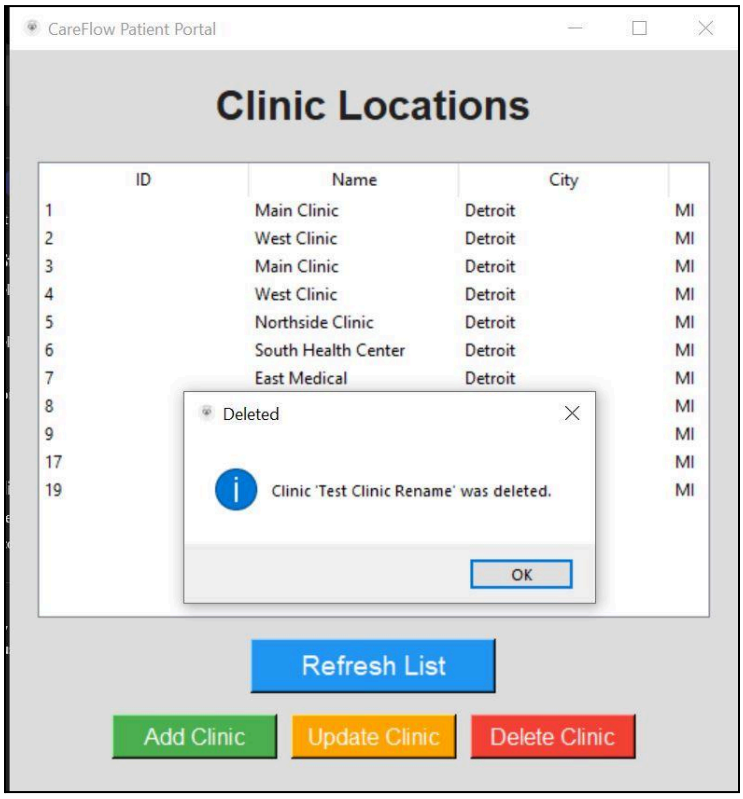
Select location to delete



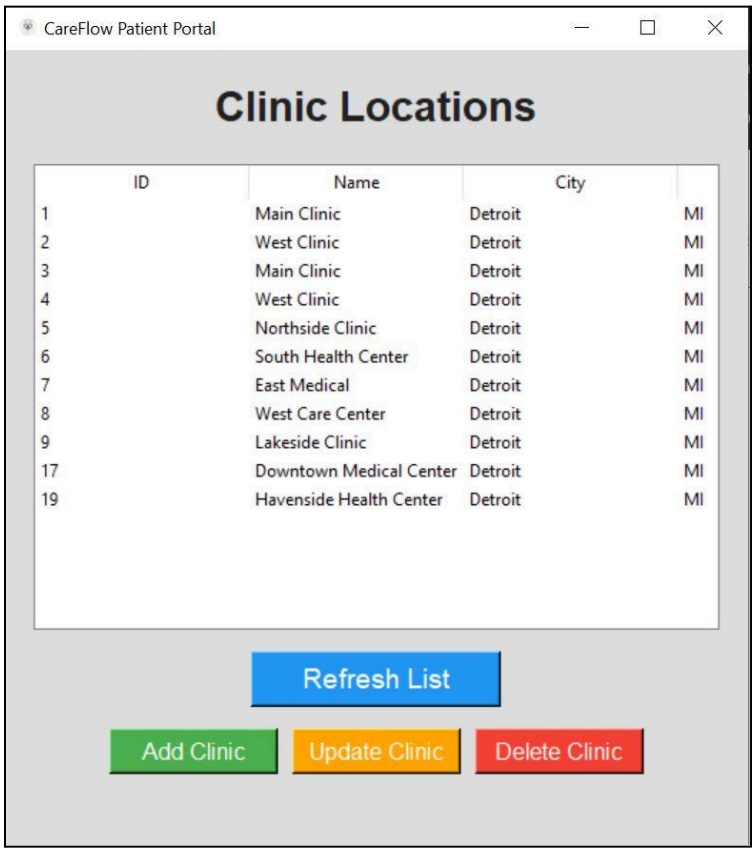
Confirm deletion prompt



Deletion Confirmation prompt



Updated display



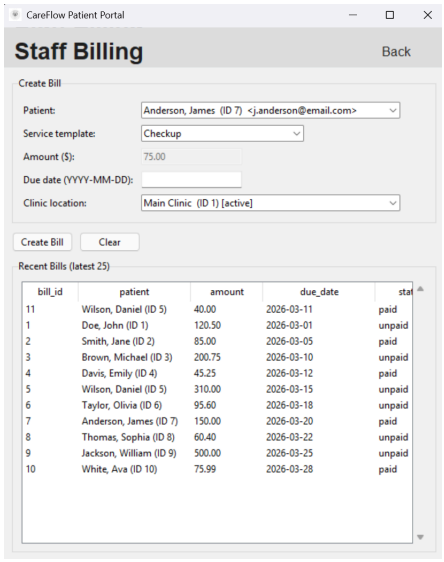
Use Case 8: Add Bills

Description: This use case allows staff to create a new bill for a specific patient at a specific clinic location. The staff selects the patient, enters the amount, selects the service type, and sets a due date. The bill is saved to the Bill table with a status of 'unpaid'.

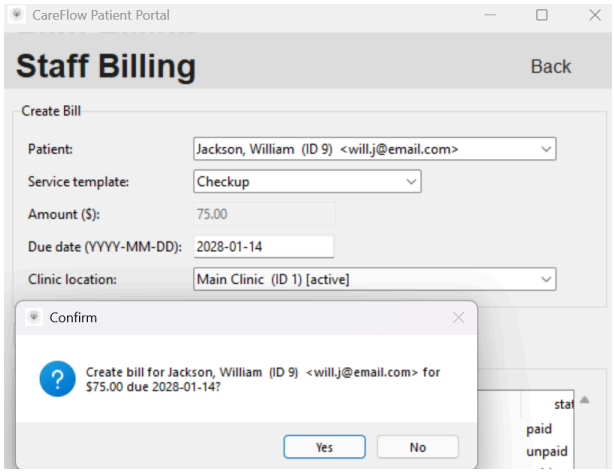
Process:

1. Staff navigates to the Billing page
2. Staff clicks the "Create Bill" button
3. A form appears prompting for patient, location, amount, and due date
4. Staff fills in all required fields
5. Staff clicks submit
6. System saves the bill to the database
7. System displays a confirmation message and refreshes the bill list

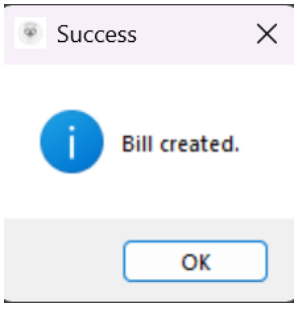
Staff Clicks Create Bill



Staff fills in bill details:



Confirmation message:



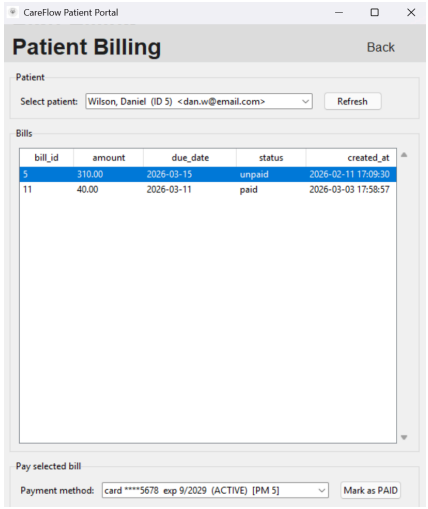
Use Case 9: Pay Bill

Description: This use case allows a patient to pay an outstanding bill using a saved payment method. The patient selects an unpaid bill, selects their payment method, and confirms the payment. The system updates the bill status to 'paid' in the database.

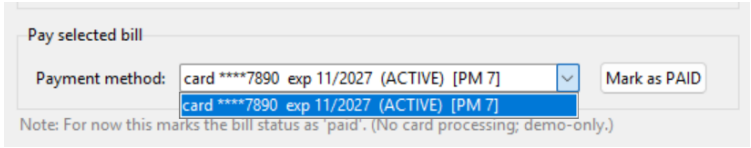
Process:

1. Patient navigates to the Billing page
2. System displays a list of bills associated with the patient's ID
3. Patient selects an unpaid bill from the list
4. Patient selects a payment method from the dropdown
5. Patient clicks "Mark as Paid"
6. System updates the bill status to 'paid' in the Bill table
7. System refreshes the bill list and displays a success message

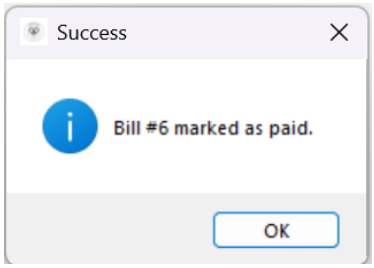
Patient Selects Unpaid bill:



Patient Selects Payment Method



Bill Marked as paid:



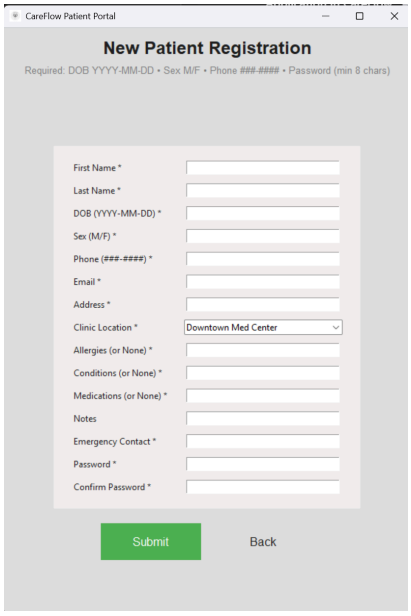
Use Case 10: Add Patient

Description:
This use case allows staff or admin to register a new patient in the system. The patient's personal, medical, and login information is collected through a registration form and stored securely in the Patient table.

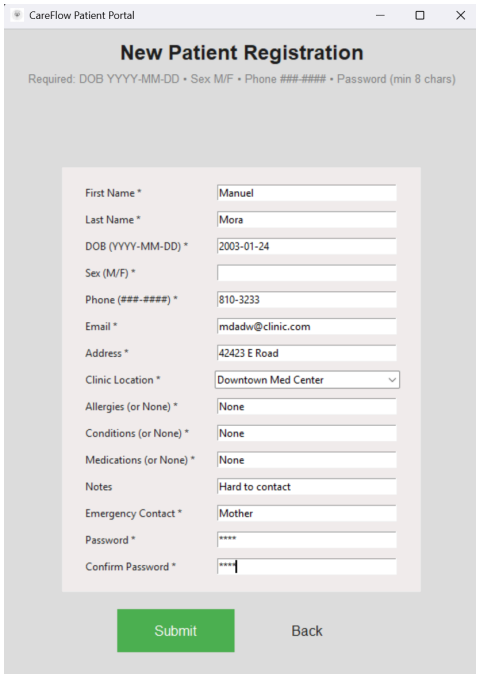
Process:

1. Staff clicks "Add Patient" from the Patient Management page
2. System displays the New Patient Registration form
3. Staff enters required fields: First Name, Last Name, DOB, Sex, Phone, Email, Address, Clinic Location, Allergies, Conditions, Medications, Emergency Contact, and Password
4. Staff clicks Submit
5. System validates all required fields
6. System hashes the password and inserts the record into the Patient table
7. System displays "Patient added successfully" and returns to the patient list

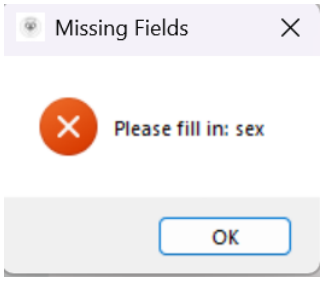
Staff Navigates to Patient Registration:



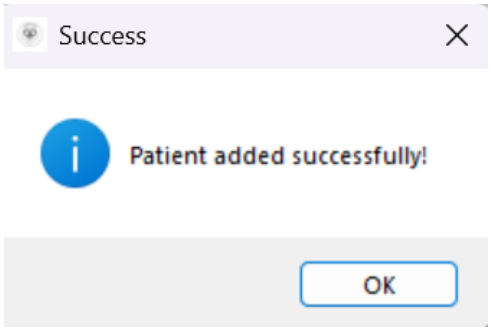
Sample Data



Error Example:



Success Message:



Use Case 11: Add Staff — Mara Evans

Description:

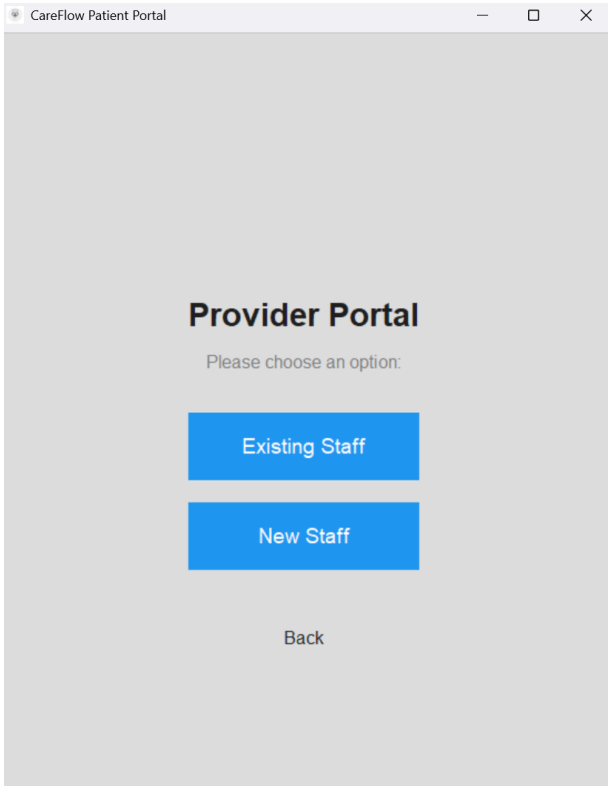
This use case allows an admin to register a new staff member in the system. The staff member's personal information, role, and credentials are collected through a registration form. A staff confirmation code is required as a security measure to prevent unauthorized

account creation.

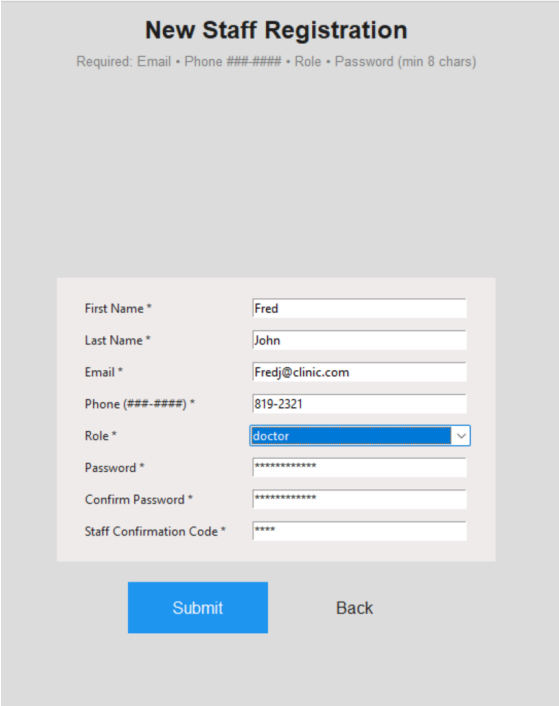
Process:

1. Admin navigates to the Staff Management page
2. Admin clicks "Add New Staff"
3. System displays the New Staff Registration form
4. Admin enters First Name, Last Name, Email, Phone, Role, Password, Confirm Password, and Staff Confirmation Code
5. Admin clicks Submit
6. System validates the confirmation code against the stored value
7. System hashes the password and inserts the record into the Staff table
8. System displays "Staff member added successfully" and returns to the staff list

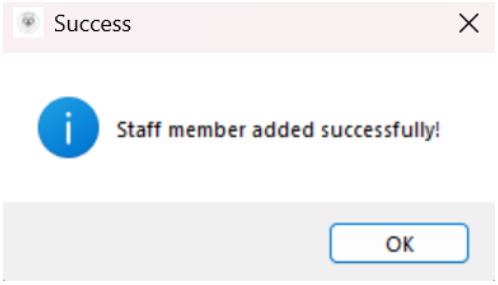
Admin Clicks Add New Staff:



Sample Data With Role:

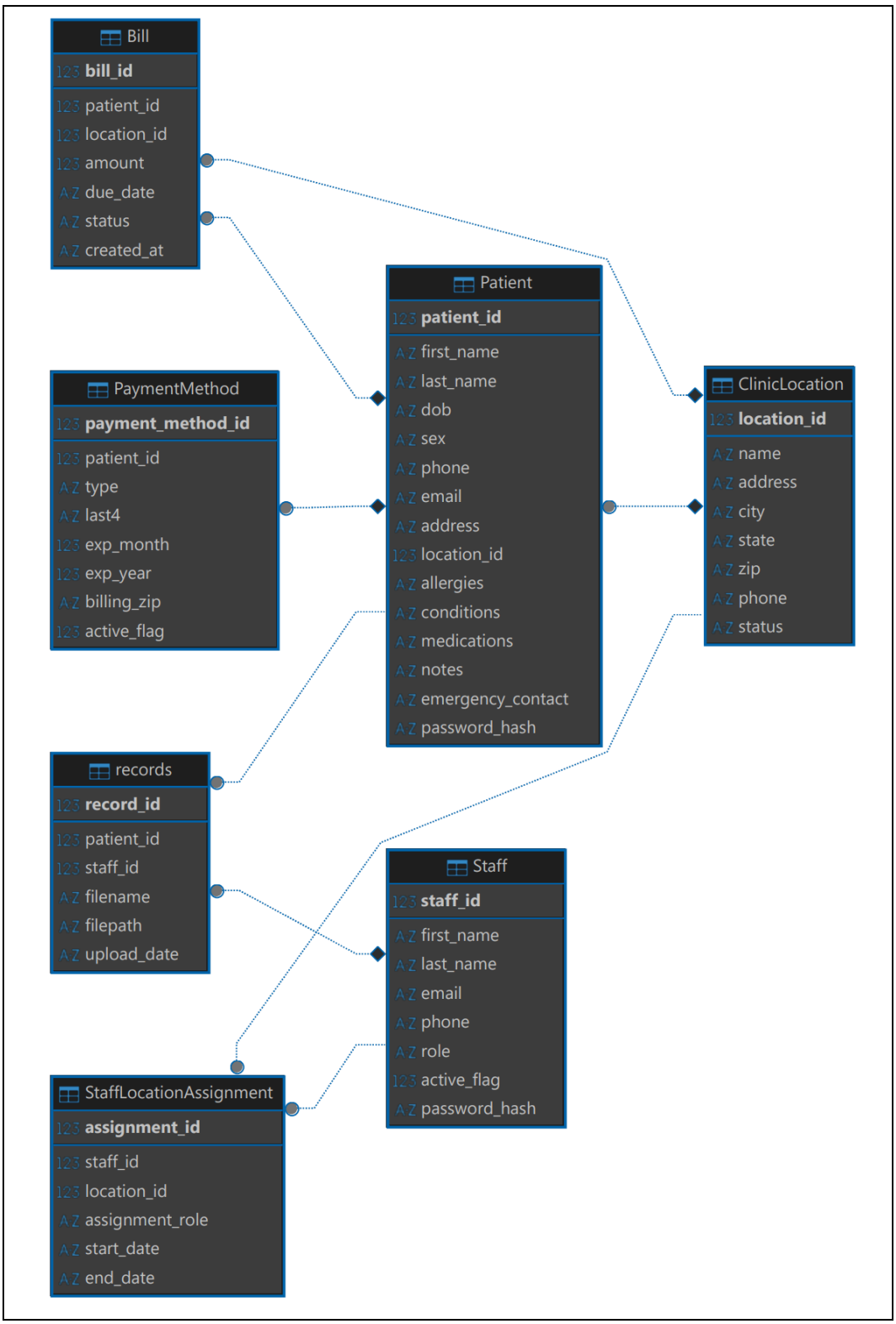


Staff Succesfully Added:



Database Design:

ERD:



Database Tables:

ClinicLocation:

location_id	name	address	city	state	zip	phone	status
1	Main Clinic	123 Health St	Detroit	MI	48201	555-1111	active
2	West Clinic	456 Care Ave	Detroit	MI	48202	555-2222	active
3	Main Clinic	123 Health St	Detroit	MI	48201	555-1111	active
4	West Clinic	456 Care Ave	Detroit	MI	48202	555-2222	active
5	Northside Clinic	111 North Rd	Detroit	MI	48203	555-3001	active
6	South Health Center	222 South Blvd	Detroit	MI	48204	555-3002	active
7	East Medical	333 East Ave	Detroit	MI	48205	555-3003	active
8	West Care Center	444 West Dr	Detroit	MI	48206	555-3004	active
9	Lakeside Clinic	555 Lake Shore	Detroit	MI	48207	555-3005	active
10	Downtown Med Center	Detroit	MI	MI	48226	555-6789	inactive
11	Test Clinic	123 Test St	Detroit	MI	48201	555-1234	inactive
12	Havenside Clinic	1234 North St	Detroit	MI	48272	''	inactive
13	Havenside	1234 Cloud Ln	Detroit	MI	47007	''	inactive
14	Havenside Clinic	1234 Cloud Ln	Detroit	MI	47007	''	inactive
15	Havenside Clinic	1234 Cloud Ln	Detroit	MI	47007	''	inactive
16	Haven Medical Center	1234 Altlas Ln	Detroit	MI	47008	''	inactive
17	Downtown Medical Center	1234 Altas Ln	Detroit	MI	47006	''	active
18	Havenside Clinic	1236 Cloud Ln	Detroit	MI	47008	''	inactive
19	NorthEast Clinic	50345	Troy	MI	48043	123-1234	inactive
20	Troy Clinic	434 E	Detriot	MI	3214	833-3213	active
21	Field Clinic	Rochester AVE	Rochester	MI	4331	313-344	active
22	Favorite Health	32133 E ave	Macomb	MI	48023	312-3243	active
23	Fresh Health	3234 E Road	Oakland	MI	6753	232-3323	active
24	Enhanced Health	32131233 Road	Detriot	MI	3213	546-6789	active
25	Healthier Clinic	4321 E	Detriot	MI	4321	432-1323	active
26	Thirty Clinic	3121 E road	Red	MI	2123	545-2311	active
27	Twenty8 Clinic	Healthy Rd	Detriot	MI	41223	312-3213	active
28	First Health	Cloud LN	Detriot	MI	31312	313-3213	active
29	Last Health	3213 E road	Macomb	MI	3213	565-3133	active
30	MockUP Clinic	312321 E road	Macomb	MI	54321	321-3313	active

Bill:

bill_id	patie...	locati...	amount	due_date	status	created_at
1	1	1	120.5	2026-03-01	unpaid	2026-02-11 17:09:30
2	2	2	85	2026-03-05	paid	2026-02-11 17:09:30
3	3	3	200.75	2026-03-10	unpaid	2026-02-11 17:09:30
4	4	4	45.25	2026-03-12	paid	2026-02-11 17:09:30
5	5	5	310	2026-03-15	unpaid	2026-02-11 17:09:30
6	6	6	95.6	2026-03-18	unpaid	2026-02-11 17:09:30
7	7	7	150	2026-03-20	paid	2026-02-11 17:09:30
8	8	8	60.4	2026-03-22	unpaid	2026-02-11 17:09:30
9	9	9	500	2026-03-25	unpaid	2026-02-11 17:09:30
10	10	10	75.99	2026-03-28	paid	2026-02-11 17:09:30
11	5	5	40	2026-03-11	paid	2026-03-03 17:58:57

Staff:

Records:

record_id	patient_id	staff_id	filename	filepath	upload_date
2	7	NULL	P67 Health.pdf	record_files\P67 Health.pdf	2026-03-07 13:28:00
3	7	NULL	Allergies Health .pdf	record_files\Allergies Health .pdf	2026-03-07 13:28:07
4	4	NULL	Allergies Health _2.pdf	record_files\Allergies Health _2.pdf	2026-03-07 13:35:28
5	4	NULL	Patient Info_2.pdf	record_files\Patient Info_2.pdf	2026-03-07 13:35:38
6	8	NULL	Patient Info_3.pdf	record_files\Patient Info_3.pdf	2026-03-07 13:35:50
7	8	NULL	Allergies Health _3.pdf	record_files\Allergies Health _3.pdf	2026-03-07 13:35:59
8	5	NULL	Patient Info_4.pdf	record_files\Patient Info_4.pdf	2026-03-07 13:36:08
9	5	NULL	Allergies Health _4.pdf	record_files\Allergies Health _4.pdf	2026-03-07 13:36:17
10	10	NULL	Patient Info_5.pdf	record_files\Patient Info_5.pdf	2026-03-07 13:36:26
11	10	NULL	Allergies Health _5.pdf	record_files\Allergies Health _5.pdf	2026-03-07 13:36:33
12	3	NULL	Patient Info_6.pdf	record_files\Patient Info_6.pdf	2026-03-07 13:36:53
13	3	NULL	Allergies Health _6.pdf	record_files\Allergies Health _6.pdf	2026-03-07 13:37:00

Patient:

patient_id	first_name	last_name	dob	sex	phone	email	address	locati...	allergies	conditions	medicati...	notes
1	John	Doe	1990-05-10	M	555-1111	john.doe@email.com	101 Main St	1	Peanuts	Asthma	Albuterol	Prefers morning visits
2	Jane	Smith	1985-03-22	F	555-2222	jane.smith@email.com	202 Oak Ave	1	Penicillin	Diabetes	Metformin	Needs wheelchair access
3	Michael	Brown	1978-11-02	M	555-3333	mike.b@email.com	303 Pine Rd	1	None	High blood pressure	Lisinopril	Monitor BP
4	Emily	Davis	1995-07-18	F	555-4444	emily.d@email.com	404 Cedar St	1	Shellfish	None	None	First-time patient
5	Daniel	Wilson	1988-01-30	M	555-5555	dan.w@email.com	505 Birch Ln	1	Latex	Eczema	Hydrocortisone	Skin sensitivity
6	Olivia	Taylor	2001-09-12	F	555-6666	olivia.t@email.com	606 Maple Dr	1	None	Migraines	Ibuprofen	Sensitive to light
7	James	Anderson	1969-04-25	M	555-7777	j.anderson@email.com	707 Lake Rd	1	Dust	Arthritis	Naproxen	Chronic joint pain
8	Sophia	Thomas	1999-12-15	F	555-8888	sophia.t@email.com	808 River St	1	Gluten	Celiac disease	Dietary control	Strict diet
9	William	Jackson	1975-06-08	M	555-9999	will.j@email.com	909 Hill Ave	1	Bee stings	None	EpiPen	Carries EpiPen
10	Ava	White	1993-02-27	F	555-0000	ava.w@email.com	1000 Elm St	1	Cats	Anxiety	Sertraline	Prefers telehealth

PaymentMethod:

payment_method_id	patient_id	type	last4	exp_mon...	exp_year	billing_zip	active_flag
1	1	card	1234	5	2027	48201	1
2	2	card	2345	6	2026	48202	1
3	3	bank	3456	7	2028	48203	1
4	4	card	4567	8	2027	48204	1
5	5	card	5678	9	2029	48205	1
6	6	bank	6789	10	2026	48206	1
7	7	card	7890	11	2027	48207	1
8	8	card	8901	12	2028	48226	1
9	9	bank	9012	1	2029	48210	1
10	10	card	0123	2	2027	48211	1

StaffLocationAssignment:

assignment_id	staff_id	location_id	assignment...	start_date	end_date
1	1	1	Administrator	2024-01-01	NULL
2	2	2	Billing Manager	2024-02-01	NULL
3	3	3	Records Clerk	2024-03-01	NULL
4	4	4	Patient Coordinator	2024-04-01	NULL
5	5	5	Billing Specialist	2024-05-01	NULL
6	6	6	Front Desk	2024-06-01	NULL
7	7	7	Admin Assistant	2024-07-01	NULL
8	8	8	Records Manager	2024-08-01	NULL
9	9	9	Patient Support	2024-09-01	NULL
10	10	10	Clinic Manager	2024-10-01	NULL

Sample Code/Explanations:

add_clinic_location: Tessa Esman

The add_clinic_location function is responsible for adding a new clinic record to the clinic location table in the database. It provides a safe, reusable way to insert clinic information while ensuring that the database schema and connection are handled correctly. This function is typically called when a staff member or administrator wants to register a new clinic in the system.

Inputs:

- name - official name of the clinic
- address - legal street address of the clinic
- city - The city where the clinic is located
- state - The state where the clinic is located
- zip_code - The postal code for the clinic location
- phone - The clinic's contact phone number

Behavior:

1. Opens a connection to the SQLite database defined by DB_NAME.
2. Creates a cursor to execute SQL commands
3. Inserts a new row into the ClinicLocation table using a parameterized SQL statement to safely insert the provided input values. The status column is automatically set to 'active' for all new clinics.
4. Commits the transaction to save changes to the database.
5. Closes the database connection to avoid locking or leaving open resources.

Error Handling:

- If any database error occurs (constraint violations, table not found), the function catches the exception, and displays an error message using a pop-up to inform the user of the problem.
- The function then returns False to signal failure.

Return Value:

- Returns True if the clinic is successfully added.
- Returns False if the operation fails due to a database error.

Usage notes:

- Using parameterized queries helps prevent SQL injection attacks and ensures that input values are safely handled.
- Because the function handles both committing the transaction and closing the database connection, it reduces the risk of leaving the database in an inconsistent state.

```
116
117 def add_clinic_location(name, address, city, state, zip_code, phone):
118     try:
119         conn = sqlite3.connect(DB_NAME)
120         cur = conn.cursor()
121
122         cur.execute("""
123             INSERT INTO ClinicLocation (name, address, city, state, zip, phone, status)
124             VALUES (?, ?, ?, ?, ?, ?, 'active')
125         """, (name, address, city, state, zip_code, phone))
126
127         conn.commit()
128         conn.close()
129         return True
130
131     except sqlite3.Error as e:
132         messagebox.showerror("Database Error", f"Failed to add clinic:\n\n{e}")
133         return False
134
```

Figure 24

Flow Chart - Add Clinic Location

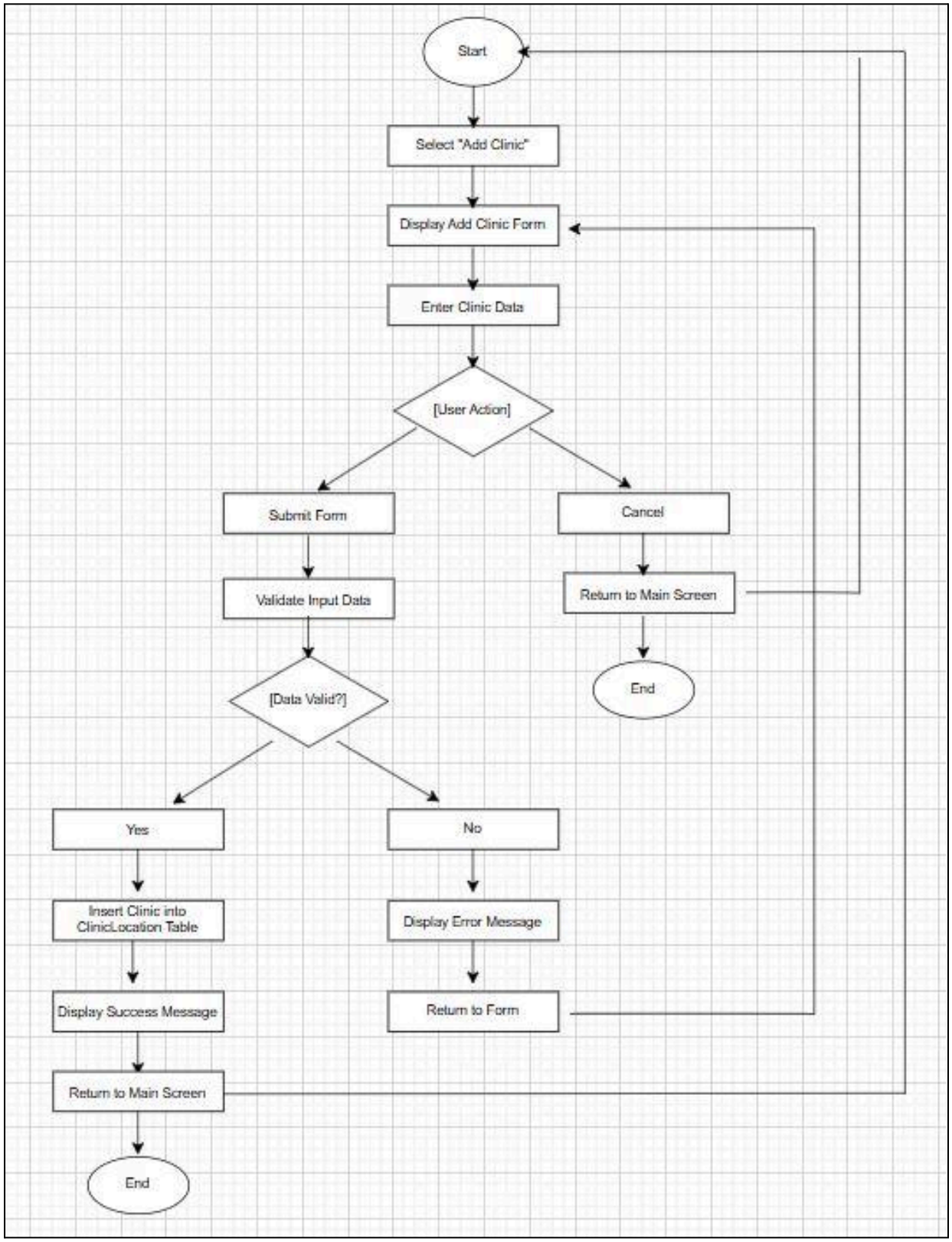


Figure 25

delete_function: Manuel Gardea

The delete_file(self function is used to remove a selected record file from the system. This includes deleting the file from the database and local storage folder where the file is saved. First, the function checks whether the user has selected a record from the table by calling the get_selected_record() helper function. This function will return the record_id and the filename of the selected item. If no file is selected, a warning message appears asking the user to select a file to delete, and the function stops running. If a record is selected, the program then saves the record_id and file so it knows exactly which record to delta. Before deleting anything the program shows a confirmation message asking the user if they are sure they want to delete the file. This will help prevent any accidental deletions. If the user chooses No, the function stops and nothing is deleted. If the user confirms the deletion the program connects to the SQLite database using the get_con() function and creates a cursor so it can execute SQL queries. The program then runs a query to retrieve the file path from the records table that matches the selected record_id. This file path tells the program where the actual file is stored on the computer. If a matching record is found the program then checks whether the file actually exists in the system using os.path.exists(). If the file exists it will delete the file from the record_files folder using os.remove(). After removing the physical file, the program deletes the corresponding record from the database using the DELETE SQL statement. The database changes are then saved using the commit(), and the database connection is closed. Finally the program will refresh the file list in the interface by calling update_file_list() so the table reflects the updated data. A success message then appears to inform the user that the file was successfully deleted. In case of any error during the process, the program catches the exceptions and displays an error message to the user. As shown in figure 26, and 27

```python
# ------------------------------
# Delete File
# ------------------------------
def delete_file(self):
    selected = self.get_selected_record()
    if not selected:
        messagebox.showwarning("Select a file", "Please select a record to delete.")
        return

    record_id, filename = selected

    confirm = messagebox.askyesno("Confirm Delete", f"Are you sure you want to delete '{filename}'?")
    if not confirm:
        return

    try:
        conn = get_conn()
        cur = conn.cursor()

        cur.execute("SELECT filepath FROM records WHERE record_id = ?", (record_id,))
        row = cur.fetchone()

        if row:
            file_path = row[0]
            if file_path and os.path.exists(file_path):
                os.remove(file_path)

        cur.execute("DELETE FROM records WHERE record_id = ?", (record_id,))
        conn.commit()
        conn.close()

        self.update_file_list()
        messagebox.showinfo("Success", f"{filename} deleted successfully.")

    except Exception as e:
        messagebox.showerror("Error", str(e))
```

Figure 26: This code was used to Delete a file from record system

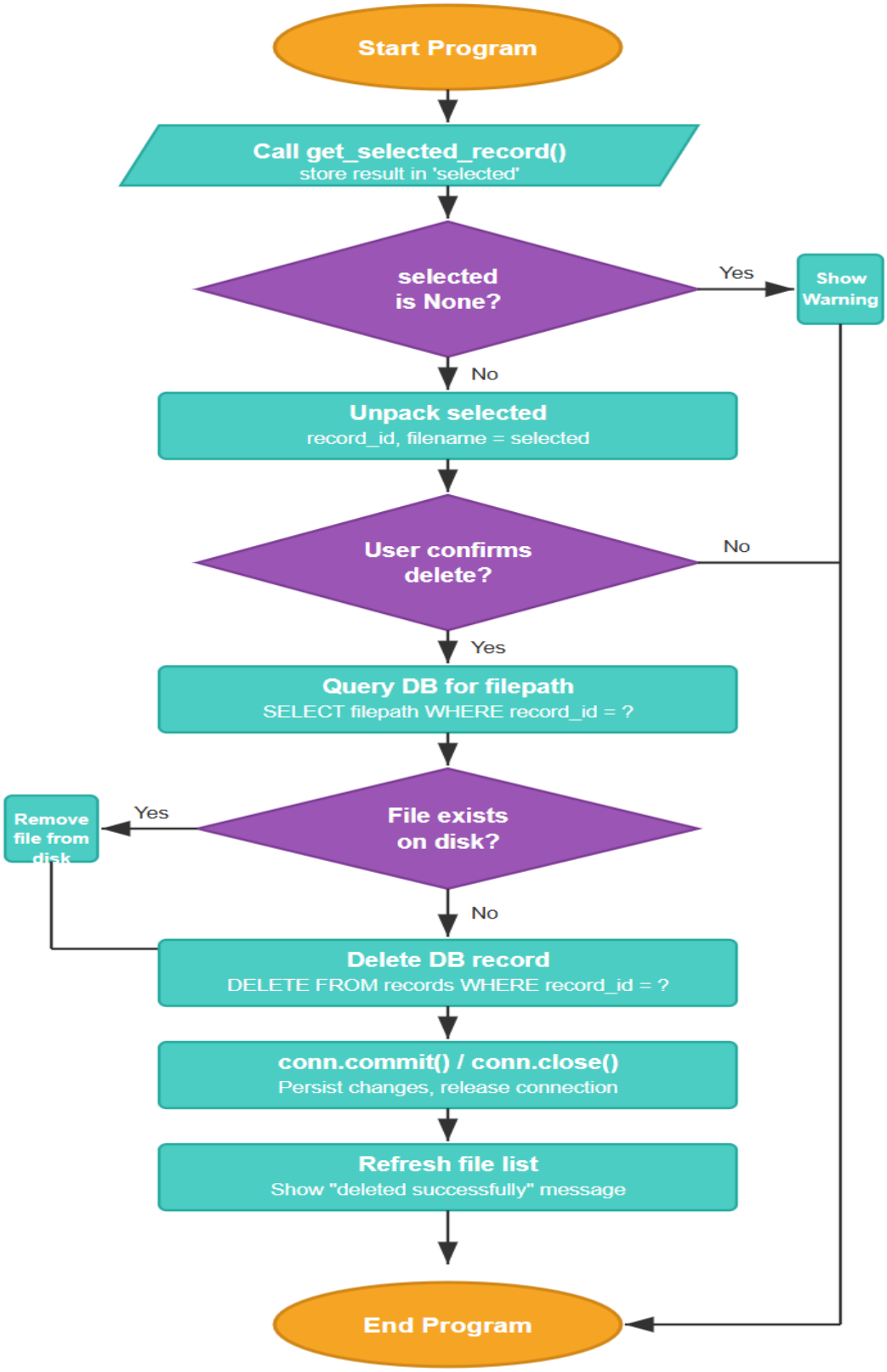


Figure 27: Flowchart of delete_file function

upload_file : Zakaria Hoque

The upload_file function will allow users to upload a file to the record system, and create a record of the file in the database table “records”. The code starts by creating a variable called file_path, whose value is the askopenfilename() method of the filedialog python library. This will open up the filesystem GUI and allow the user to select the file and upload. This is followed by an if statement. This if statement creates a condition if there is a file selected and uploaded, otherwise the condition will not run. The condition creates a filename variable, whose value is derived from the OS python library, and creates a filename based off the base name of the file_path variable (ex. If the file path is “C/:Users/Hoque/record1”, the name will be “record1”). The dest_path variable constructs a new file path, setting the file within the new record directory. This is followed by a try-except block to handle errors that may occur. Within the try block, shutil.copy(file_path,dest_path) copies the file from the original location (file_path) to the new location (dest_path). Upload_date variable uses the datetime library of python to get the current date and time of the upload action and stores this data. Following this is the cursor.execute method, which sends the following SQL command: "INSERT INTO records (filename, filepath, upload_date) VALUES (?, ?, ?)",

(filename, dest_path, upload_date).

This SQL command inserts the filename, dest_path and upload_date variable data into the filename, filepath and upload-date entries of the records table, thus allowing a way to track where a file is located and at what date and time. This is then followed by the conn.commit() method, which commits the transaction with the database to ensure it is saved. This is then followed by the self.update_file_list() method, which updates the listbox to reflect the database changes, allowing a user to see their file was uploaded in real time. The

messagebox.showinfo(“Success”, f”{filename} uploaded successfully.”) method open a messagebox popup that tells the user the file was successfully uploaded. After this is the exception code block. The exception is stored as variable e, and should any error occur, an error messagebox will appear to tell the user “error”. As shown in figure 28, and 29

```
# ==============================
# Upload File
# ==============================
def upload_file(self):
    file_path = filedialog.askopenfilename()
    if file_path:
        filename = os.path.basename(file_path)
        dest_path = os.path.join(DIRECTORY, filename)

        try:
            shutil.copy(file_path, dest_path)

            upload_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

            cursor.execute(
                "INSERT INTO records (filename, filepath, upload_date) VALUES (?, ?, ?)",
                (filename, dest_path, upload_date)
            )
            conn.commit()

            self.update_file_list()
            messagebox.showinfo("Success", f"{filename} uploaded successfully.")

        except Exception as e:
            messagebox.showerror("Error", str(e))
```

Figure 28, This code was used to upload a file to the record system

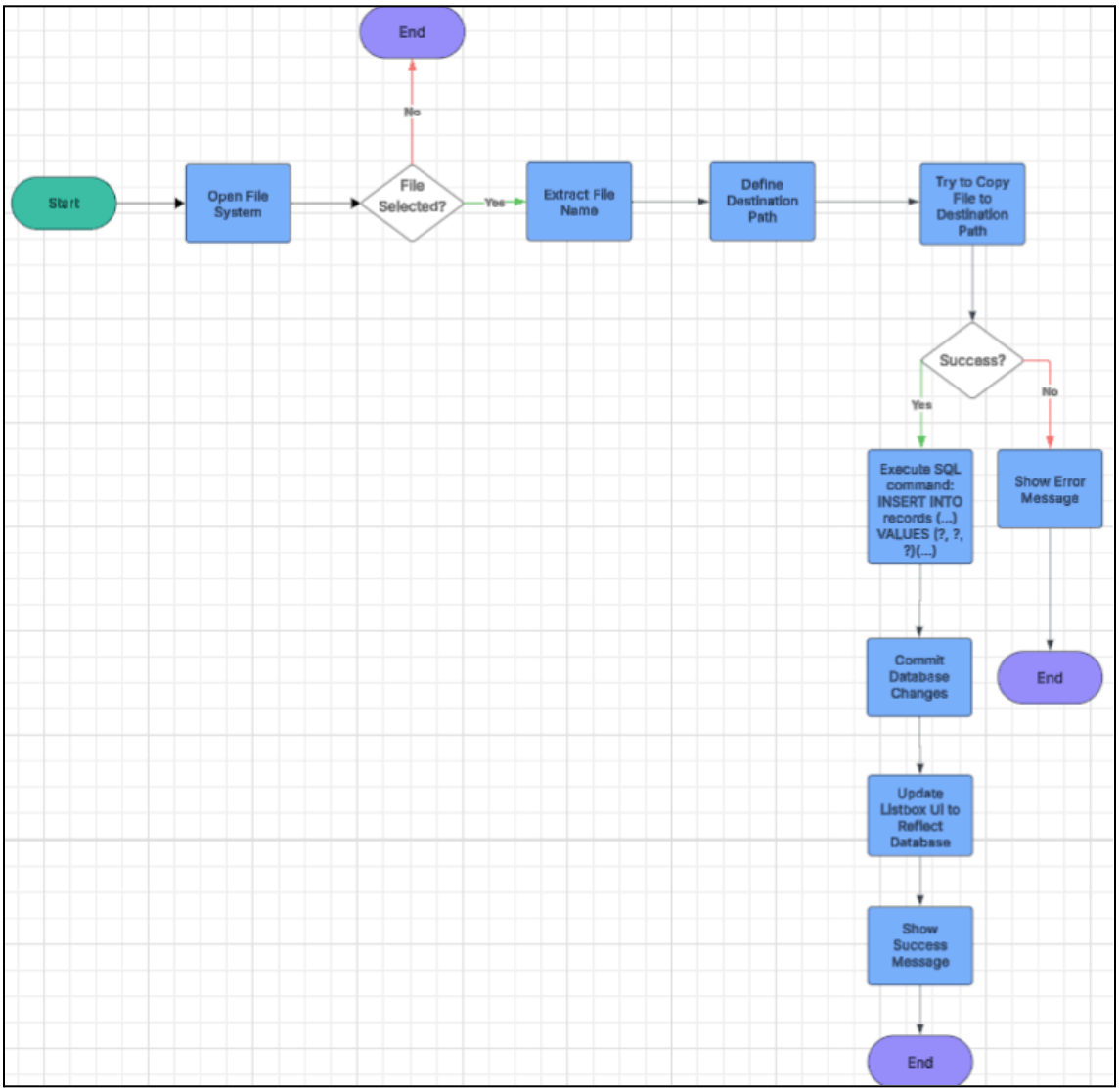


Figure 29, Flowchart of upload_file function

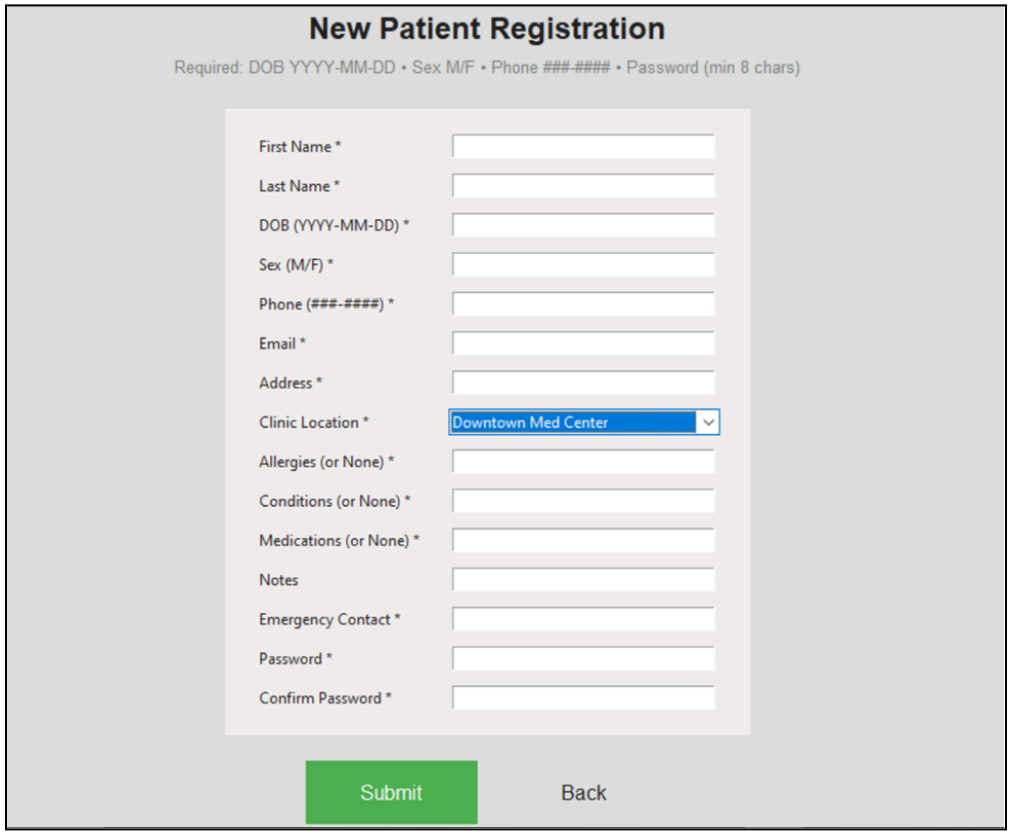
Add New Patient: Logan Templin

The following SQLite command is responsible for inserting a new patient record into the CareFlow database. The command uses the INSERT INTO statement to add data to the Patient table. Each column listed in the statement corresponds to a specific piece of patient information collected from the registration form, including personal information, contact information, medical details, and authentication data. Once the command executes successfully, the database commits the information using conn.commit(),storing the new patient record in the database. Finally, the database connection is closed using conn.close(). The data gathered from the user is all text input based, beside clinic

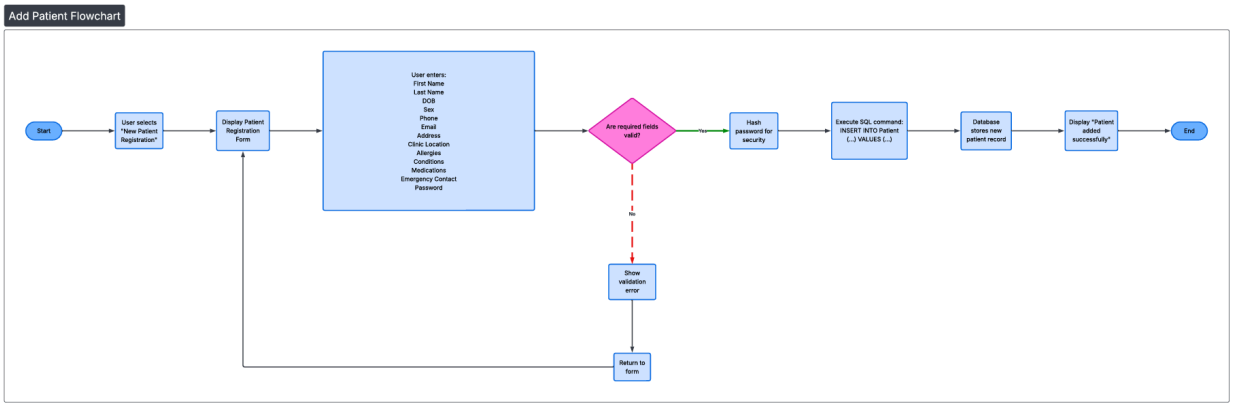
location, which rather than manual input, the user must select a location from a dropdown list. As shown in figure 30

```
try:
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute("""
        INSERT INTO Patient (
            first_name, last_name, dob, sex, phone, email, address,
            location_id, allergies, conditions, medications,
            notes, emergency_contact, password_hash
        ) VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?)
    """, (
        data["first_name"],
        data["last_name"],
        data["dob"],
        data["sex"],
        data["phone"],
        data["email"],
        data["address"],
        location_id,
        data["allergies"],
        data["conditions"],
        data["medications"],
        data.get("notes", ""),
        data["emergency_contact"],
        pw_hash
    ))
    conn.commit()
    conn.close()
```

Figure 30



Flow Chart (Add new patient):



Add New Staff: Mara Evans

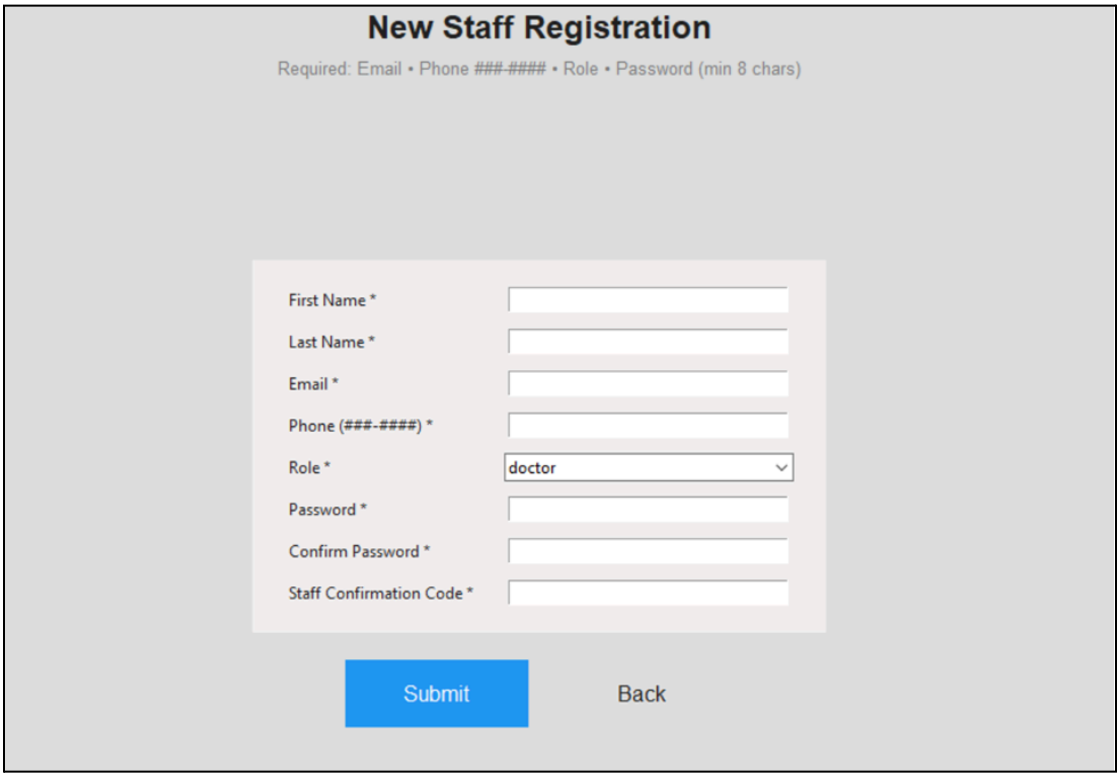
The following SQLite command is responsible for inserting a new staff member into the Staff table of the CareFlow database. The INSERT INTO statement used to store the staff member's information, including their first name, last name, email address, phone number, role, and password hash. The query also sets the active_flag value to 1, which means that the staff member's account is active by default when created(a staff member being active indicates they can be assigned to clinic locations). The VALUES section uses placeholders (?) that are replaced with data collected from the staff registration form in the application. These values are stored in the data and are passed into the SQLite command when the query is executed. The password is stored as a hashed value (pw_hash) for security purposes rather than storing the plain text password. After the SQL command executes successfully, the database transaction is saved using conn.commit(), and the connection to the database is closed with conn.close(). Similar to the patient page this form is also primarily text input based, the role selection for staff contain a drop down with specific staff jobs (like doctor). The last field is the staff confirmation code, this does not get logged to the database, it is a security feature to ensure patients cannot assign themselves as staff members. In order to create a new staff you need the specific confirmation number to pass (1234).

```
try:
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()

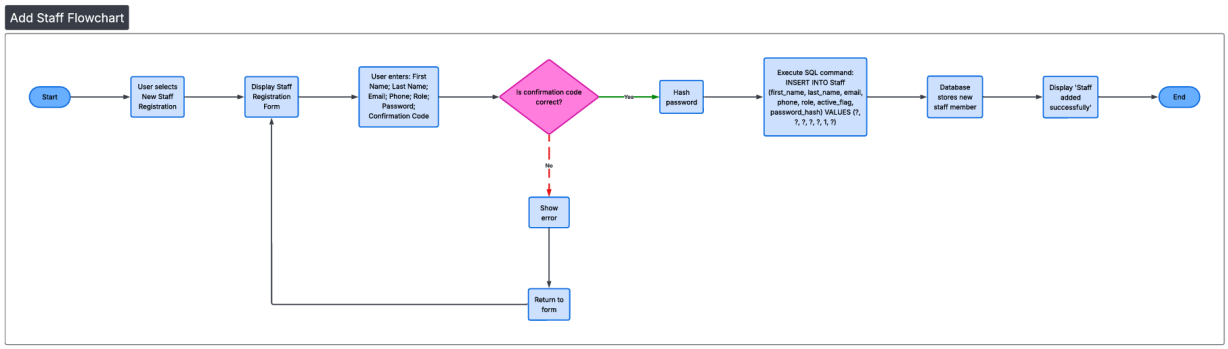
    cur.execute("""
        INSERT INTO Staff (first_name, last_name, email, phone, role, active_flag, password_hash)
        VALUES (?, ?, ?, ?, ?, 1, ?)
    """, (
        data["first_name"],
        data["last_name"],
        data["email"],
        data["phone"],
        role,
        pw_hash
    ))

    conn.commit()
    conn.close()
```

Try a statement to add the new staff information to the database.



New staff registration portal



Flowchart (Add New Staff)

Future Update:

Patient Portal Development: As of now, we had to zero in on staff/admin features rather than patient/provider like we had originally planned, meaning that the patient side may get benched. This would be great for future work, after completing the provider side it would be made accessible to patients as well.

Some workflow changes: Want to enable features to streamline approval processes and allow cross-clinic data sharing (approval required for security).

New Features: Ensure mobile and responsive design, login functionality, improved interface and personalization, favorites feature.

Appendix A: System Vision Document
System Name: CareFlow
Project Type: Healthcare Management Web Application
Course: CSI 4999 - Senior Capstone Project
Term: Winter 2026

Problem Description:
Many healthcare organizations still heavily rely on paper-based workflows or multiple disconnected systems to manage clinic locations, staff, patient information, billing, and medical records. These processes make it difficult to keep data up to date and accurate, especially when managing multiple clinic locations. Staff can often spend unnecessary time searching for information, duplicating work, or correcting errors caused by incorrect information.

As healthcare organizations continually grow, these issues are becoming more and more significant. There is a need for a centralized system that allows clinics to manage administrative and operational tasks efficiently while still ensuring secure handling of sensitive patient information. A single integrated system will improve accessibility, reduce paperwork, and better support coordination across multiple clinic locations.

System Capabilities
The Careflow system should be capable of the following:

- Managing multiple clinic locations within a single system
- Assigning and managing staff across different clinic locations
- Creating, updating, and maintaining patient information/records
- Storing and managing patient medical information and documents
- Generating and managing patient medical information and documents
- Processing payments and tracking billing status
- Uploading, searching, and downloading medical documents
- Enforcing role-based access so users will only see features that are relevant to their role

Business Benefits
It is anticipated that the deployment of CareFlow will provide the following benefits:

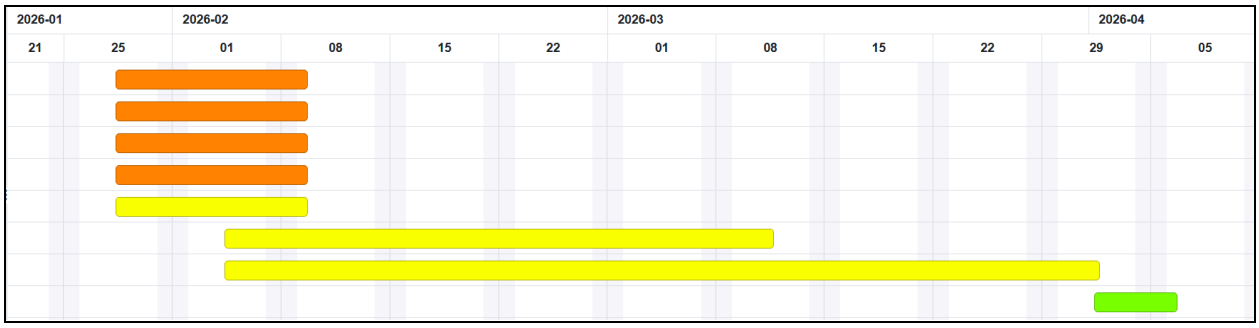
- Reduce reliance on paper-based processes and manual record keeping
- Improved data accuracy and consistency across multiple clinic locations
- Increase efficiency for administrative staff by centralizing operations
- Improve accessibility to patient, billing, and document information
- Better coordination between staff, billing, and records departments
- Provide an affordable/scalable solution that can grow with healthcare organizations

Appendix B: Work Breakdown Structure and Gantt Chart:

Work Breakdown Structure

1. Discover and understand the details of all aspects of the problem.
   a. Identify and define system functions. ~ 2 hours
   b. Identify and define use cases. ~ 2 hours
   c. Develop workflows and descriptions for the use cases. ~ 3 hours
2. Design the components of the solution to the problem.
   a. Design wireframes (input screens, output screens). ~ 5 hours
   b. Design and build databases (attributes, keys, indexes). ~ 4 hours
   c. Design overall architecture. ~ 4 hours
   d. Design program details. ~ 4 hours
3. Build the components and integrate everything into the solution.
   a. Code and unit test GUI programs. ~ 15 hours
   b. Code and unit test logic programs. ~ 30 hours
4. Perform all system-level tests and then deploy the solution.
   a. Perform final GUI functionality tests. ~ 2 hours
   b. Perform final system functionality tests. ~ 2 hours

Task Name	ID	Start	End	Duration
Decide on system fun…	1	2026-01-28	2026-02-09	9 days
Decide on and define …	2	2026-01-28	2026-02-09	9 days
Develop workflows	3	2026-01-28	2026-02-09	9 days
Design wireframes	4	2026-01-28	2026-02-09	9 days
Build database	5	2026-01-28	2026-02-09	9 days
Make GUIs	6	2026-02-04	2026-03-11	26 days
Make back end	7	2026-02-04	2026-04-01	41 days
Final functionality tests	8	2026-04-01	2026-04-06	4 days



VIDEO LINK: https://youtu.be/LS3n6_oGxro

References

Satzinger, J. W., Jackson, R. B., & Burd, S. D. (2016). *Systems analysis and design in a changing world* (7th ed.). Cengage Learning. ISBN: 978-1-305-11720-4

Python Software Foundation. (n.d.). *tkinter — Python interface to Tcl/Tk*. Python 3 Documentation. https://docs.python.org/3/library/tkinter.html

Python Software Foundation. (n.d.). *sqlite3 — DB-API 2.0 interface for SQLite databases*. Python 3 Documentation. https://docs.python.org/3/library/sqlite3.html

GeeksforGeeks. (n.d.). *Python GUI – Tkinter*. https://www.geeksforgeeks.org/python/python-gui-tkinter/

Abraham, A. (n.d.). *Flowcharts for programmers*. Medium. https://medium.com/@amanuelabraham0202/flowcharts-for-programmers-2aff6a6d8f63

NYNPTEL. (n.d.). *Understanding activity diagrams: A visual guide to system process modeling*. Medium. https://medium.com/@nynptel/understanding-activity-diagrams-a-visual-guide-to-system-process-modeling-ee46a0759e10

Tinelli, C. (2015). *Chapter 10: System sequence diagrams* [Course notes]. University of Iowa. https://homepage.cs.uiowa.edu/~tinelli/classes/022/Spring15/Notes/chap10.pdf